

TERRAFORM CHALLENGE - 03

Start working on the below challenge.

Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using Terraform. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- The service will be a simple web server that returns a static page.
- You can use **apache2** or **nginx** to serve a simple HTML file.
- The service should be **containerized** and stored in a **Docker registry**.
- The service should be deployed on a **single VM instance** as a docker container.
- The image should be pulled from the registry specified earlier.
- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

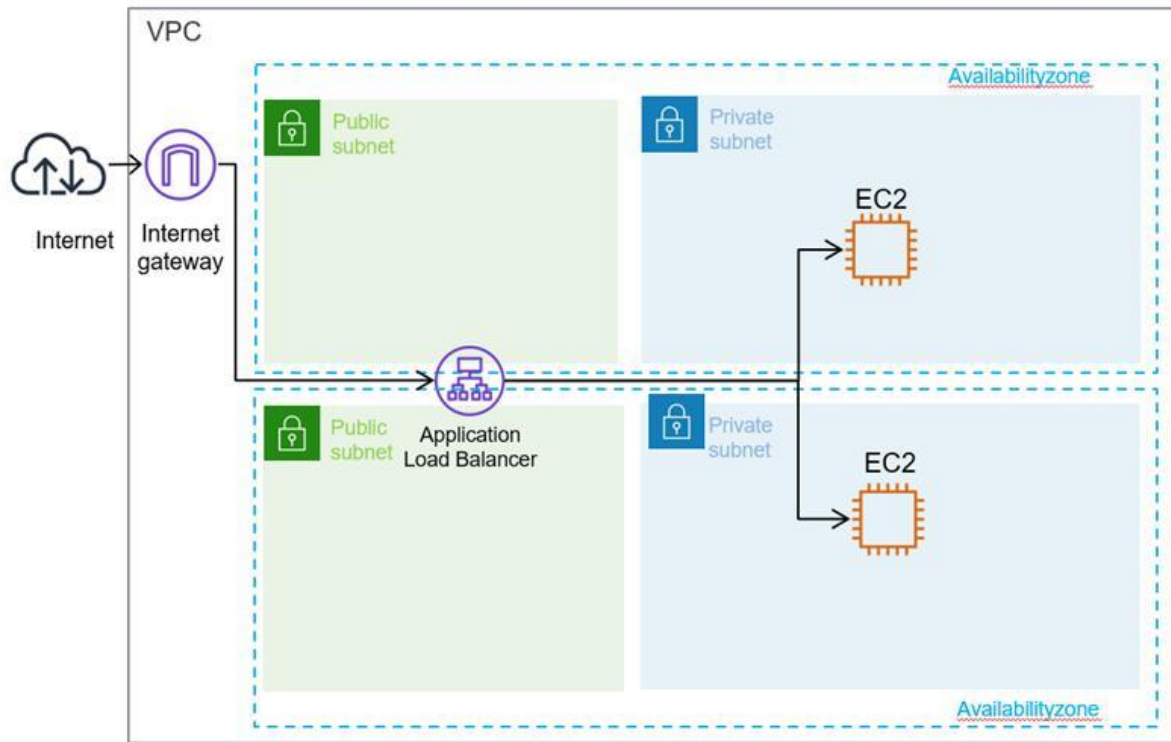
You are to provide the following as part of your solution:

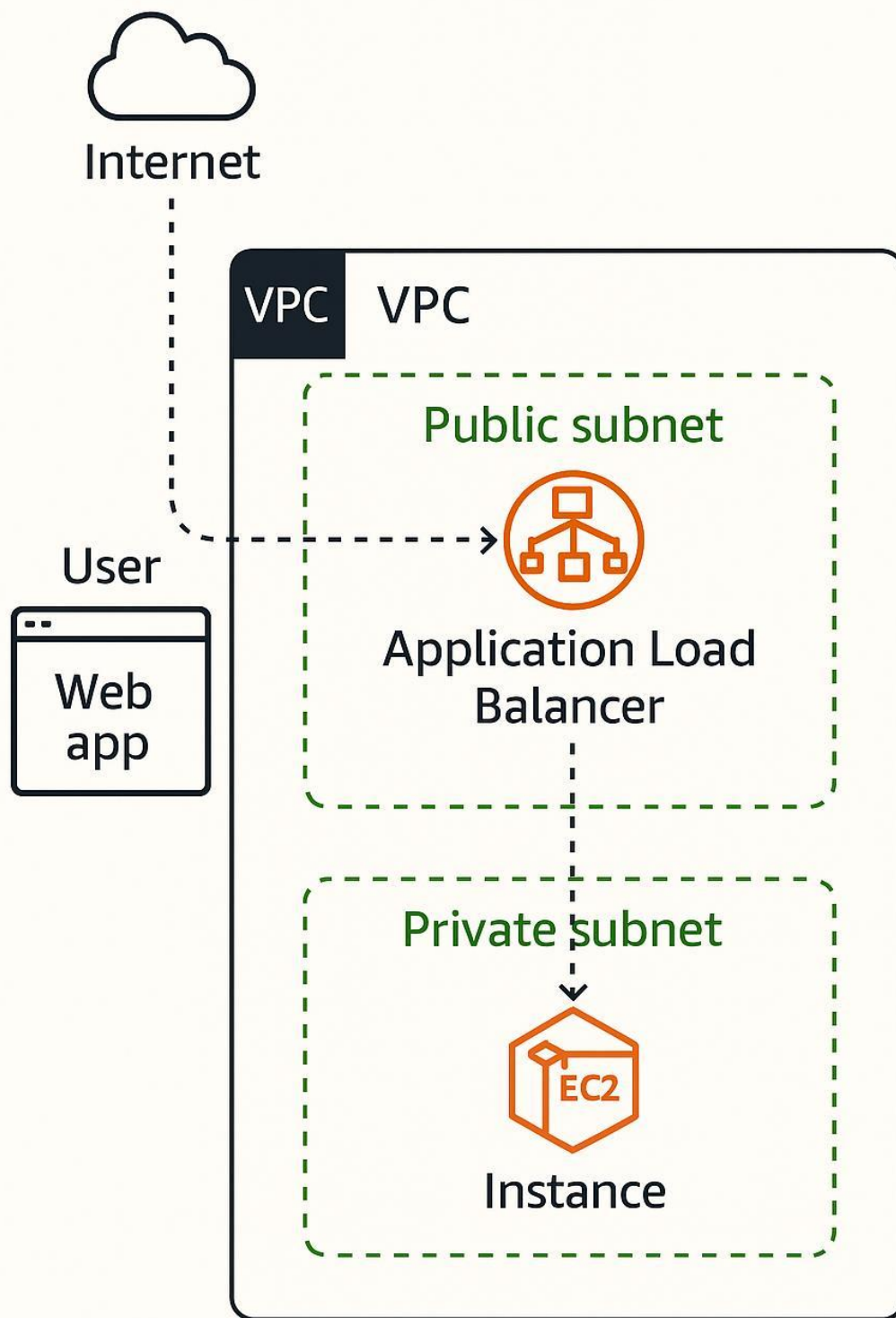
1. **Dockerfile** to build the image.
2. **Terraform codebase** to deploy the following:
 - Docker registry
 - VPC and all required networking resources
 - EC2 instance
 - Load Balancer

Notes

- Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.

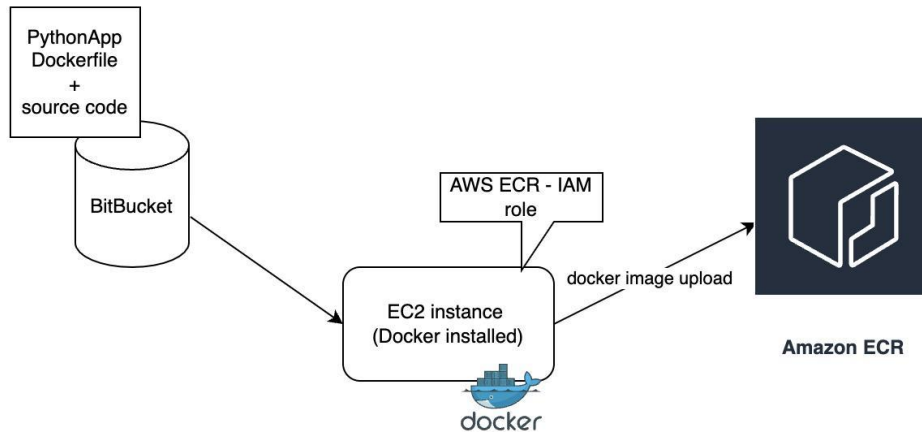
- To push the image to the Docker registry, **build the image locally** before running the Terraform code.
- Once the image registry is created, use Terraform to **push the local image** to the registry.
- Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.
- For SSH access:
 - Use a locally created key.
 - Add it to the cloud provider key storage.
 - Attach it to the instance.
- Use **Terraform best practices** for file structure and code organization.
 - Use variables, locals, modules, etc. when appropriate.
- Include all tfvars, scripts, and other helper files used to deploy the application.
- You may use any cloud provider of your choosing, although **AWS is preferred**.
- Use a **free tier account** or a **trial account** to deploy the resources. tell me step by step process with explanation



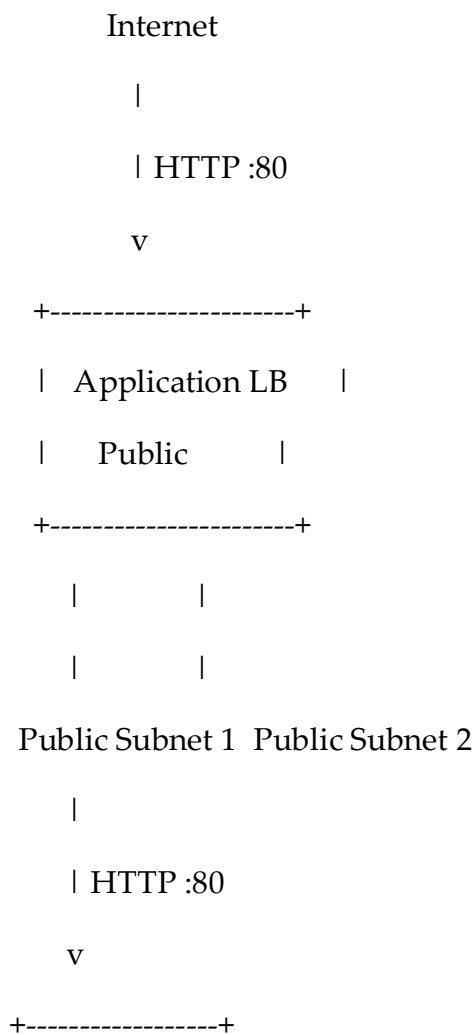


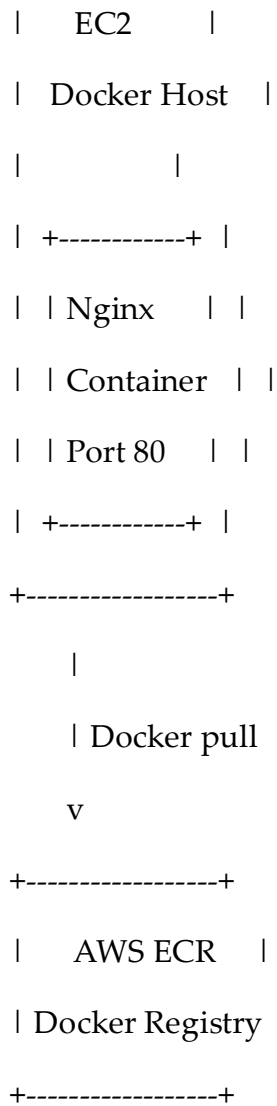
Web application diagram

Create Docker image & Push into Amazon ECR



Target architecture





SSH :22

Your Public IP

|

v

EC2

The important security point is:

Internet ---> ALB ---> EC2:80 ☒

Internet ---> EC2:80 ☐

Your IP ---> EC2:22 

Other IP ---> EC2:22 

An Application Load Balancer needs subnets in at least two Availability Zones, so we will create two public subnets.

For this lab, I would **not use a NAT Gateway**. NAT Gateway charges can make a supposedly free-tier lab unnecessarily expensive. Instead, the EC2 instance will be in a public subnet but its security group will **not allow HTTP from the Internet**. It will only allow HTTP from the ALB security group.

AWS ECR currently provides 500 MB/month of private-repository storage for one year to new ECR customers under its Free Tier, subject to AWS's current Free Tier terms.

1. What we are going to build

We will create:

Resource	Purpose
VPC	Isolated AWS network
Internet Gateway	Internet connectivity
2 public subnets	ALB requires 2 AZs
Route table	Internet routing
ECR repository	Docker registry
IAM role	EC2 can pull from ECR
IAM instance profile	Attach role to EC2
EC2	Docker host
Security Group - ALB	Allows HTTP
Security Group - EC2	Allows HTTP only from ALB + SSH from your IP
Application Load Balancer	Public entry point
Target Group	Points ALB to EC2

Resource	Purpose
Listener	HTTP :80
Key Pair	SSH access
Docker container	Runs nginx

2. Project structure

Create the following directory:

```

terraform-microservice/
|
├─ app/
|   ├── Dockerfile
|   └─ index.html
|
├─ modules/
|   ├── network/
|   |   ├── main.tf
|   |   ├── variables.tf
|   |   └─ outputs.tf
|   |
|   ├── security/
|   |   ├── main.tf
|   |   ├── variables.tf
|   |   └─ outputs.tf
|   |
|   ├── ecr/
|   |   ├── main.tf
|   |   ├── variables.tf
|   |   └─ outputs.tf
|   |
|   ├── compute/
|   |   ├── main.tf
|   |   ├── variables.tf
|   |   └─ outputs.tf
|   |
|   └─ alb/
|       ├── main.tf
|       ├── variables.tf
|       └─ outputs.tf
|
├─ main.tf
├─ variables.tf
├─ outputs.tf
├─ versions.tf
├─ terraform.tfvars
├─ user_data.sh
├─ push-image.sh
├─ .gitignore
└─ README.md

```

This is a good interview-friendly structure because networking, security, registry, compute and load balancing are separated into modules.

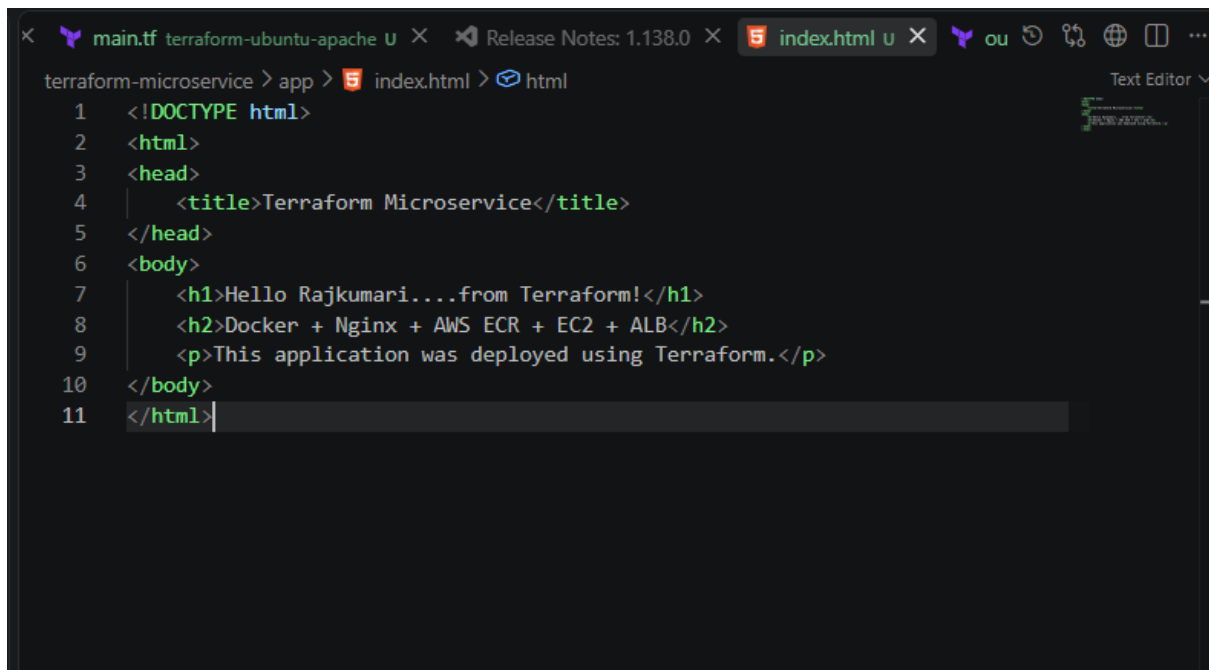
3. Step 1 — Create the application

Go into your project:

```
mkdir -p terraform-microservice/app
```

cd terraform-microservice

Create: app/index.html

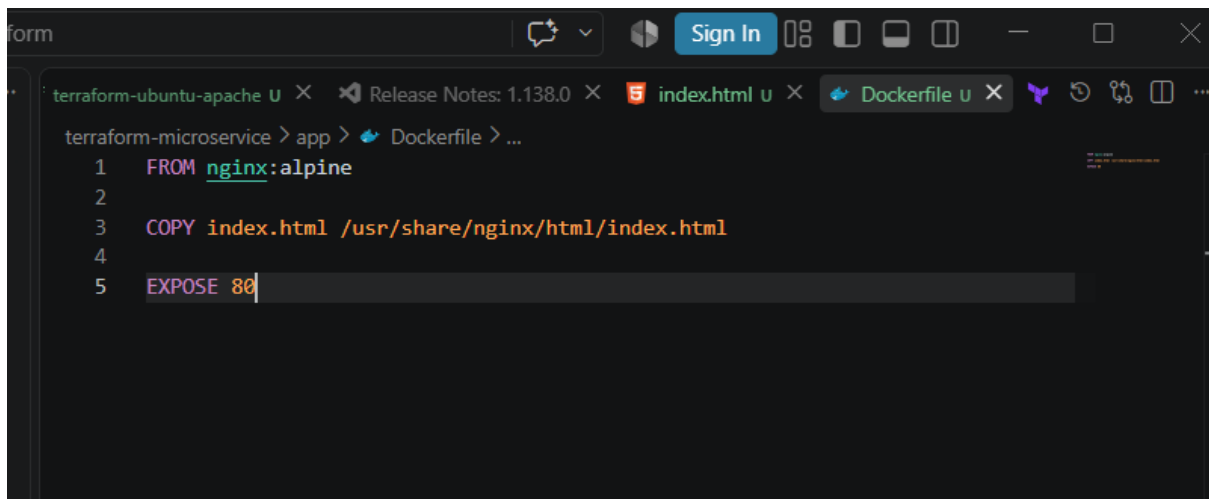
A screenshot of a code editor window. The top bar shows several tabs: 'main.tf', 'terraform-ubuntu-apache U', 'Release Notes: 1.138.0', 'index.html u', and 'ou'. The active tab is 'index.html u'. The editor content shows a file path 'terraform-microservice > app > index.html > html' and a list of 11 lines of HTML code. The code is as follows:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Terraform Microservice</title>
5 </head>
6 <body>
7   <h1>Hello Rajkumari...from Terraform!</h1>
8   <h2>Docker + Nginx + AWS ECR + EC2 + ALB</h2>
9   <p>This application was deployed using Terraform.</p>
10 </body>
11 </html>
```

4. Step 2 — Create Dockerfile

Create:

app/Dockerfile

A screenshot of a code editor window. The top bar shows several tabs: 'main.tf', 'terraform-ubuntu-apache U', 'Release Notes: 1.138.0', 'index.html u', and 'Dockerfile u'. The active tab is 'Dockerfile u'. The editor content shows a file path 'terraform-microservice > app > Dockerfile > ...' and a list of 5 lines of Dockerfile code. The code is as follows:

```
1 FROM nginx:alpine
2
3 COPY index.html /usr/share/nginx/html/index.html
4
5 EXPOSE 80
```

Use:

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Explanation

FROM nginx:alpine

Uses lightweight nginx.

COPY index.html /usr/share/nginx/html/index.html

Copies our static HTML page into nginx's default web directory.

EXPOSE 80

Documents that nginx listens on port 80.

```

MINGW64:/c/Terraform/terraform-microservice
bec5b214f0e2b99409e30653a125b37557da
#6 extracting sha256:cfc08d7798ef23f7c95fd70b79738968e8e09c82a24b3a6d6f93db5bd9a
c660d 0.0s done
#6 extracting sha256:ff72fbf3f58024d1f0273c12922d62a1f0d4c078a19a6481103cd626814
a0375 done
#6 extracting sha256:c73463be665b58abe96e6781adc9e3bd8f566852d90513766829bc0b845
98f4e 0.0s done
#6 extracting sha256:c40008edd3507e27a1c31d9015083fafcd5c5805a9bc0f91e45cfacееeb
3ffb5 0.0s done
#6 extracting sha256:edd6b88cb27df8963f30b02e1be7fe2e4b1ce08dde2cda582343c829e5
bdc30 0.0s done
#6 extracting sha256:251741e0fff499462ca895be7e9f461f06dfdbc28c8831d665ceb10b871
7b65a
#6 extracting sha256:251741e0fff499462ca895be7e9f461f06dfdbc28c8831d665ceb10b871
7b65a 0.3s done
#6 DONE 9.7s

#7 [2/2] COPY index.html /usr/share/nginx/html/index.html
#7 DONE 0.1s

#8 exporting to image
#8 exporting layers 0.0s done
#8 exporting manifest sha256:10cc16f75c81243ef27a431a86f8266b12bd87bfe9d58841209
315c32bc5d365 done
#8 exporting config sha256:5bcb9af4a9bc13d95658081a33dab3eb608bd09fee3121f2bbb8
b041e6a9440 done
#8 exporting attestation manifest sha256:5ff532372c7cf28da1016929f3a9ff8a3ce6f1d
97f45560bea7b2edf9a436cbb 0.0s done
#8 exporting manifest list sha256:f01c1f9b52421d3eb1d244287b74ec30830bf1a8914d0b
4b5b6040f995eb8416 done
#8 naming to docker.io/library/terraform-microservice:latest done
#8 unpacking to docker.io/library/terraform-microservice:latest 0.0s done
#8 DONE 0.1s

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ docker images
IMAGE                                ID                                  DISK USAGE   CONTENT SIZE  EXTRA
hello-world:latest                  96498ffd522e                        25.9kB       9.49kB        U
nginx:latest                        dd0e25651318                       253MB       69.2MB
terraform-microservice:latest       f01c1f9b5242                       102MB       28.8MB

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ docker run -d \
> --name terraform-test \
> -p 8080:80 \
> terraform-microservice:latest
8ec5c10ef3ae179c289e059db5a34894d2fe179c42bde40e57d23bfddf9c154f

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS          NAMES
8ec5c10ef3ae   terraform-microservice:latest       "/docker-entrypoint..." 26 seconds ago
Up 25 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  terraform-test

```

5. Step 3 — Build the Docker image locally

This is explicitly required by your assignment.

From the project root:

```
docker build -t terraform-microservice:latest ./app
```

Check:

```
docker images
```

You should see:

```
terraform-microservice latest
```

Test it locally:

```
docker run -d \  
  --name terraform-test \  
  -p 8080:80 \  
  terraform-microservice:latest
```

Check:

```
docker ps
```

Open:

<http://localhost:8080>

You should see:

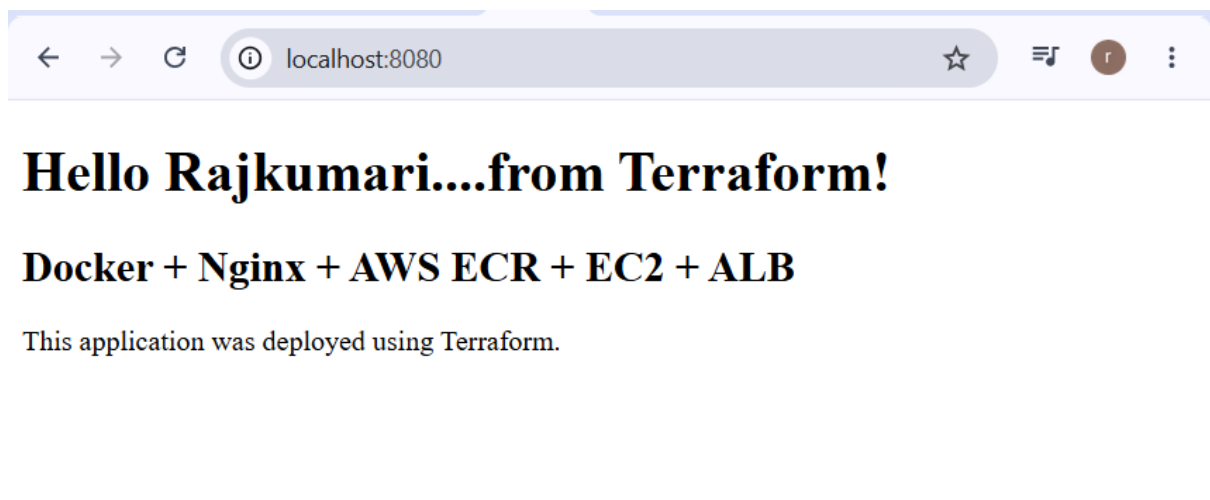
Hello from Terraform!

Docker + Nginx + AWS ECR + EC2 + ALB

This application was deployed using Terraform.

Stop it:

```
docker rm -f terraform-test
```



6. Step 4 — Create your SSH key

Do this on your **local Windows machine**, preferably Git Bash.

```
cd ~/Downloads
```

Generate a key:

```
ssh-keygen -t ed25519 -f terraform-microservice-key
```

When asked for passphrase, you can press Enter for this lab.

You will get:

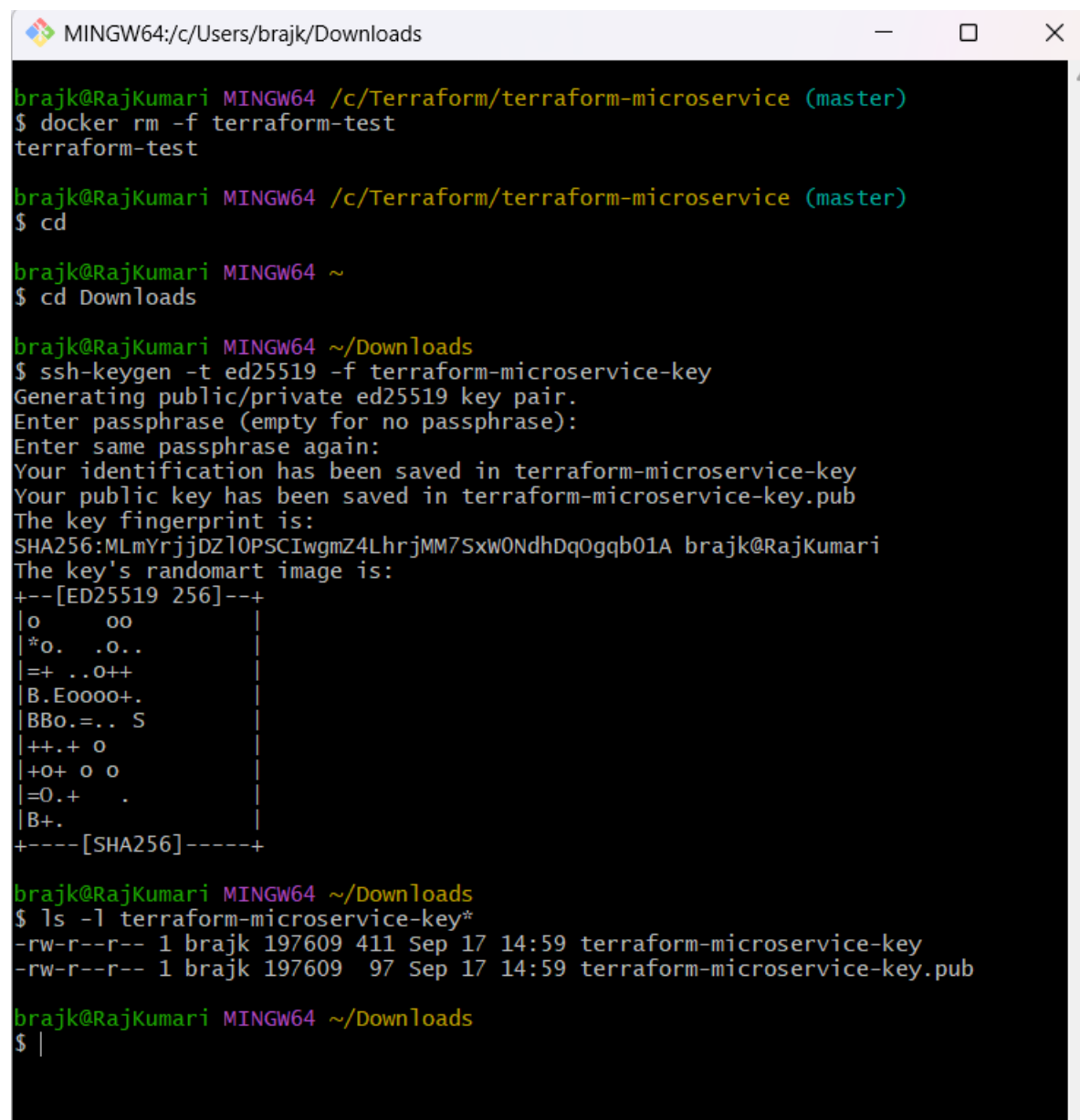
```
terraform-microservice-key
```

```
terraform-microservice-key.pub
```

Check:

```
ls -l terraform-microservice-key*
```

Never commit the private key to Git.



```
MINGW64:/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ docker rm -f terraform-test
terraform-test

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ cd

brajk@RajKumari MINGW64 ~
$ cd Downloads

brajk@RajKumari MINGW64 ~/Downloads
$ ssh-keygen -t ed25519 -f terraform-microservice-key
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in terraform-microservice-key
Your public key has been saved in terraform-microservice-key.pub
The key fingerprint is:
SHA256:MLmYrjDZl0PSCIwgmZ4LhrjMM7Sxw0NdhDqOgqb01A brajk@RajKumari
The key's randomart image is:
+---[ED25519 256]---+
|o      oo      |
|*o.   .o..     |
|=+  ..O++      |
|B.Eoooo+.      |
|BBo.=.. S      |
|+++. o         |
|+o+ o o        |
|=O.+   .        |
|B+.            |
+-----[SHA256]-----+

brajk@RajKumari MINGW64 ~/Downloads
$ ls -l terraform-microservice-key*
-rw-r--r-- 1 brajk 197609 411 Sep 17 14:59 terraform-microservice-key
-rw-r--r-- 1 brajk 197609  97 Sep 17 14:59 terraform-microservice-key.pub

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

7. Step 5 — Find your public IP

In Git Bash:

```
curl https://checkip.amazonaws.com
```

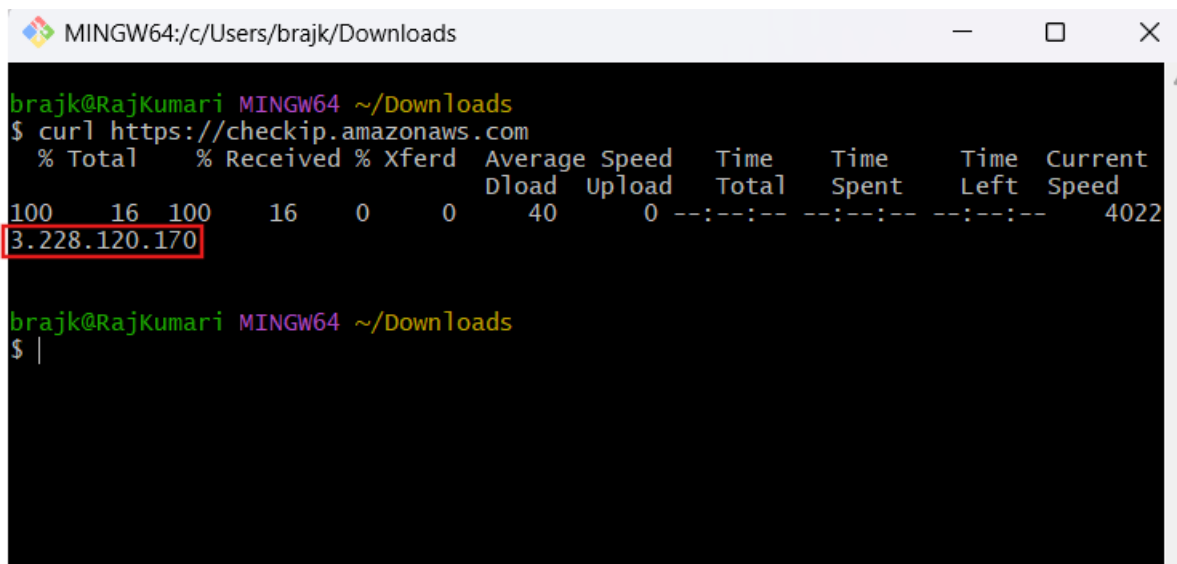
Suppose it returns:

49.205.100.25

Then your Terraform variable will be:

```
ssh_allowed_cidr = "49.205.100.25/32"
```

The /32 means exactly one IPv4 address.

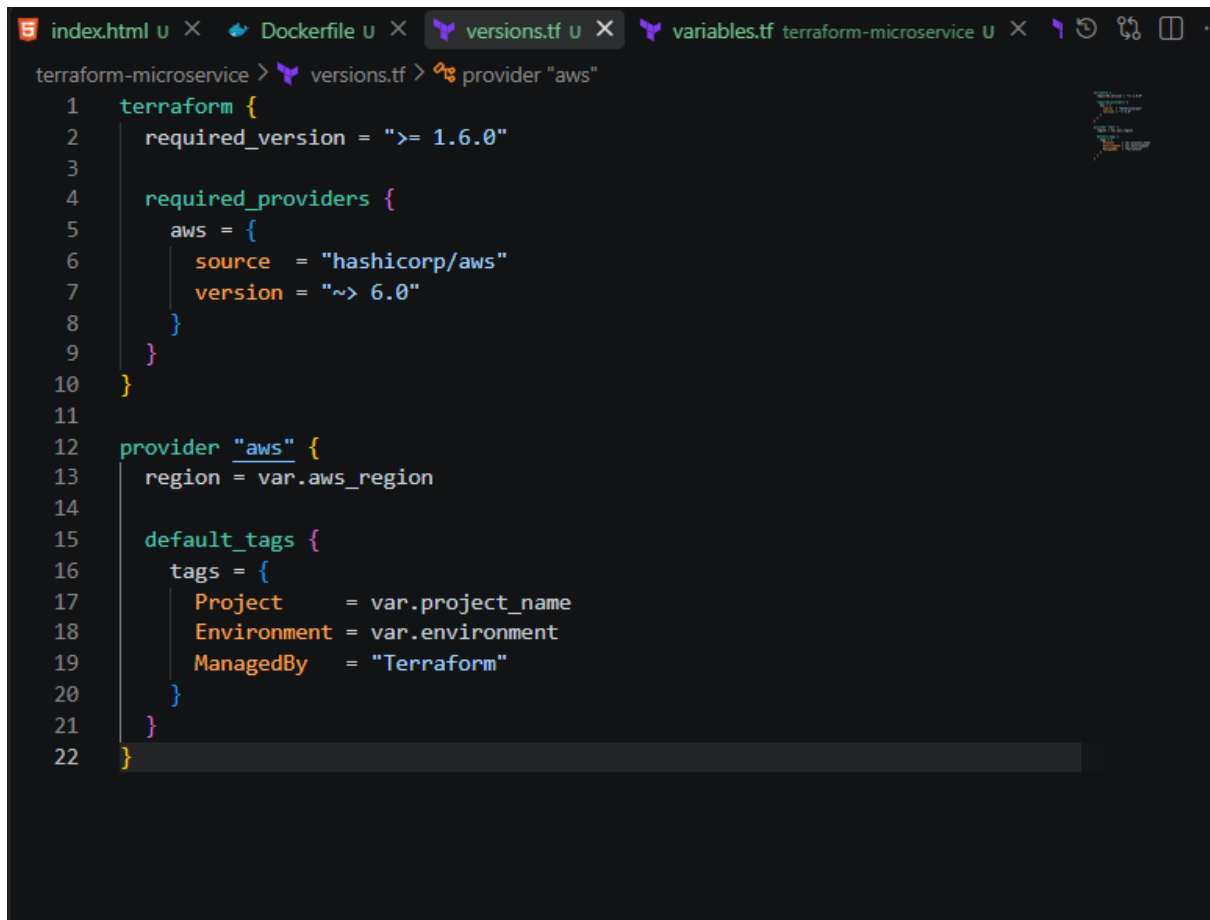


The screenshot shows a Windows Command Prompt window titled "MINGW64:/c:/Users/braj/Downloads". The prompt is "brajk@RajKumari MINGW64 ~/Downloads". The user has entered the command "\$ curl https://checkip.amazonaws.com". The output is a table showing the progress of the curl command. The first row is the header: "% Total", "% Received", "% Xferd", "Average", "Speed", "Time", "Time", "Time", "Current". The second row is the data: "100", "16", "100", "16", "0", "0", "40", "0", "--:--:--", "--:--:--", "--:--:--", "4022". The third row is the IP address: "3.228.120.170". The IP address is highlighted with a red box.

% Total	% Received	% Xferd	Average	Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left	Speed
100	16	100	16	0	0	40	0	--:--:-- 4022
3.228.120.170								

8. Step 6 — Configure Terraform

Create: versions.tf

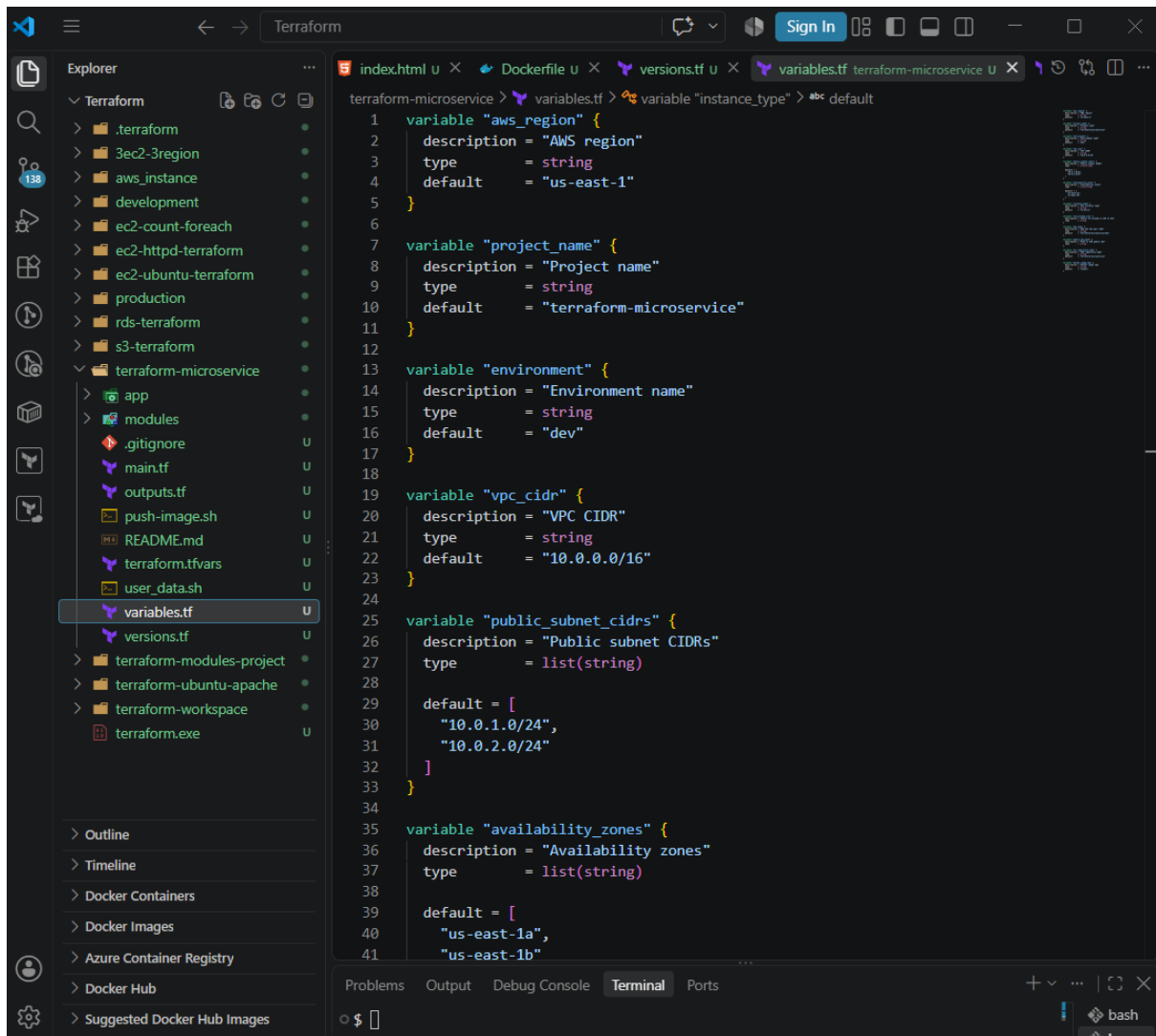
A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'index.html u', 'Dockerfile u', 'versions.tf u', 'variables.tf', and 'terraform-microservice u'. The 'versions.tf' tab is active. The code is Terraform configuration for the 'aws' provider. It includes a 'terraform' block with 'required_version' and 'required_providers' (aws), and a 'provider "aws"' block with 'region' and 'default_tags'.

```
1 terraform {
2   required_version = ">= 1.6.0"
3
4   required_providers {
5     aws = {
6       source = "hashicorp/aws"
7       version = "~> 6.0"
8     }
9   }
10 }
11
12 provider "aws" {
13   region = var.aws_region
14
15   default_tags {
16     tags = {
17       Project      = var.project_name
18       Environment  = var.environment
19       ManagedBy    = "Terraform"
20     }
21   }
22 }
```

9. Step 7 — Variables

Create:

variables.tf

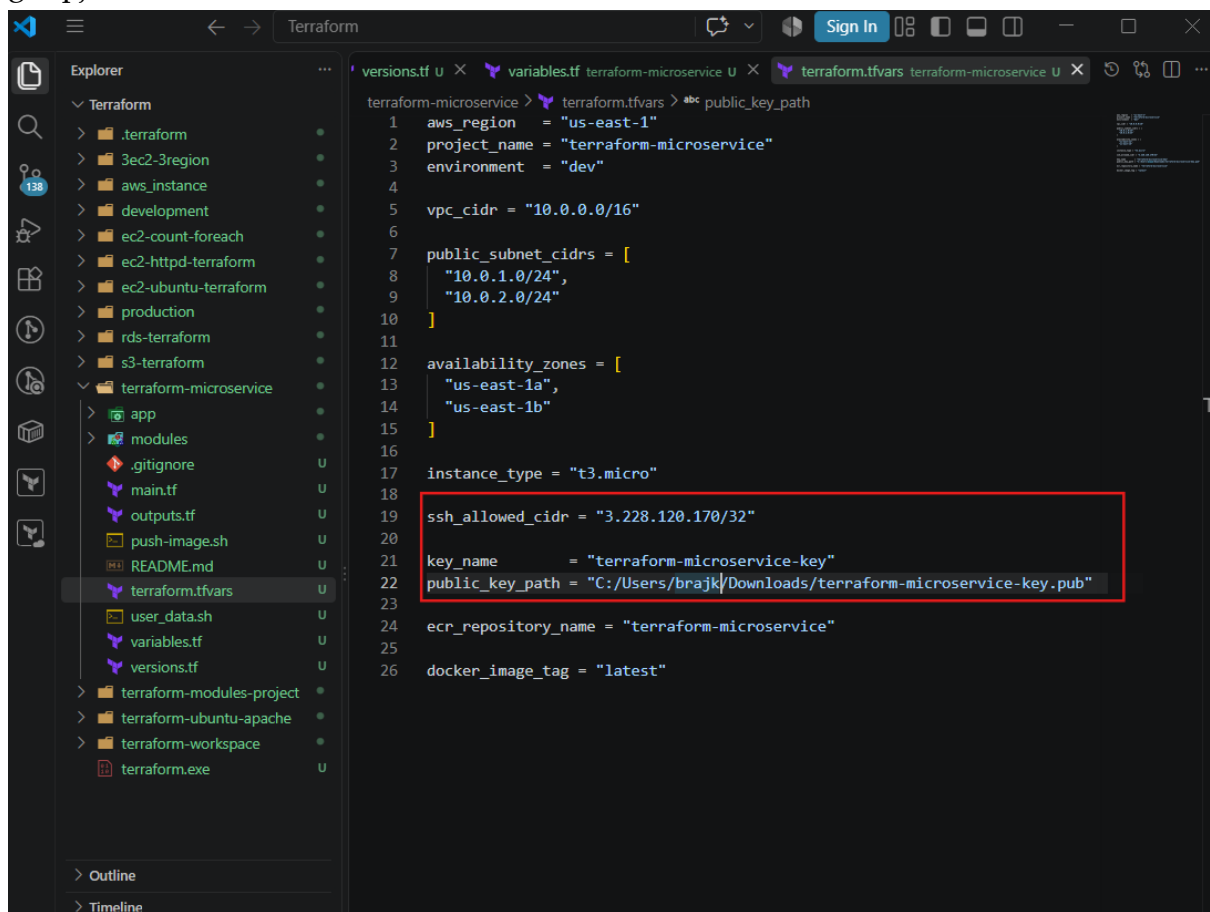


10. Step 8 — terraform.tfvars

Create:

terraform.tfvars

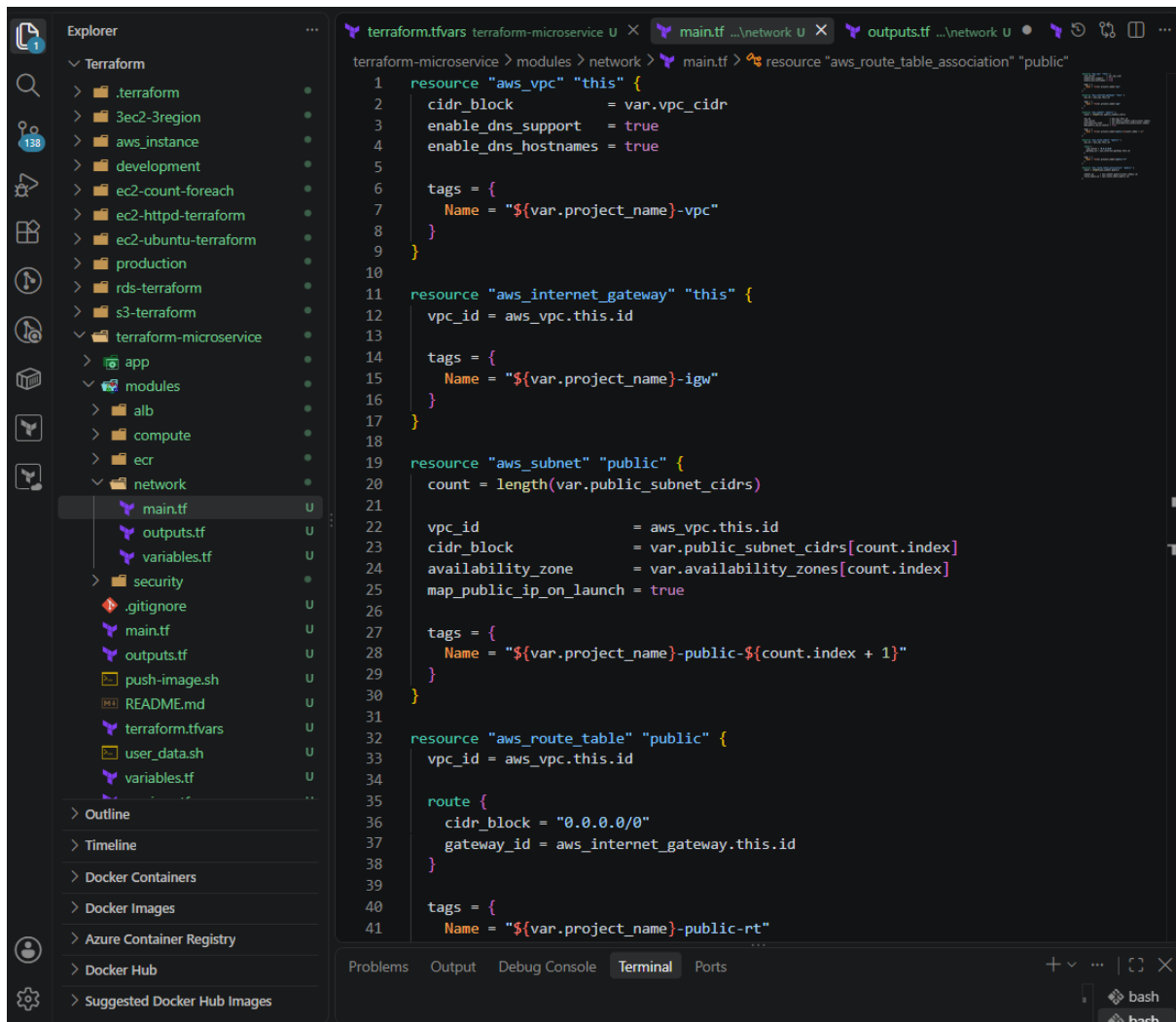
gkllpju



11. Step 9 — Network module

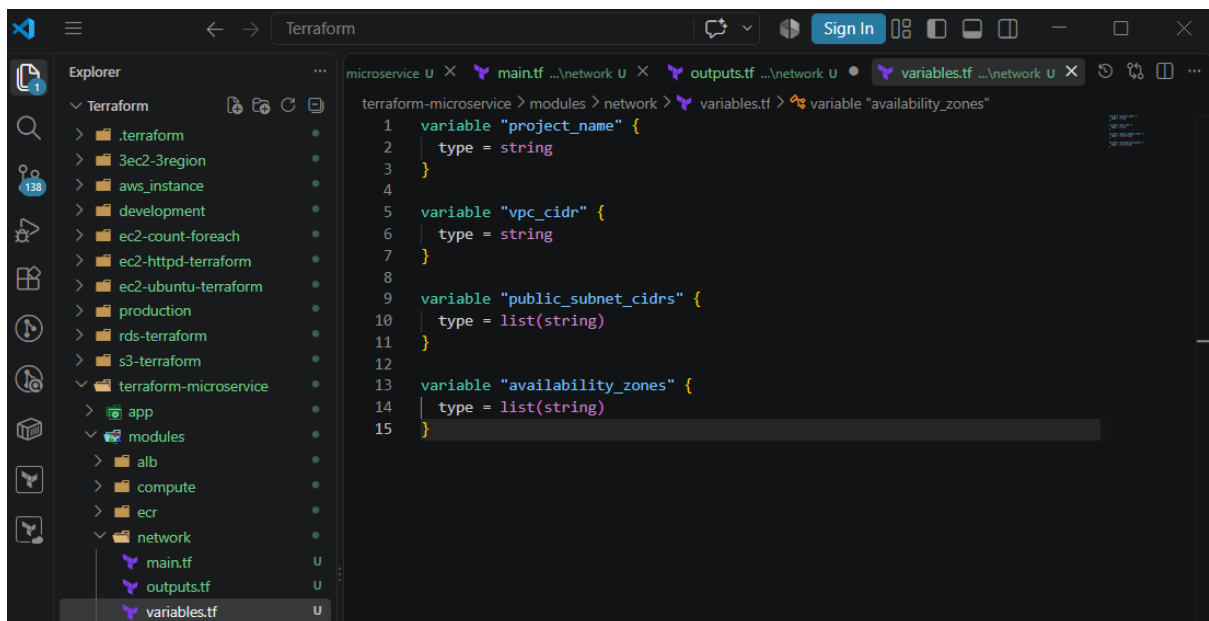
Create:

modules/network/main.tf



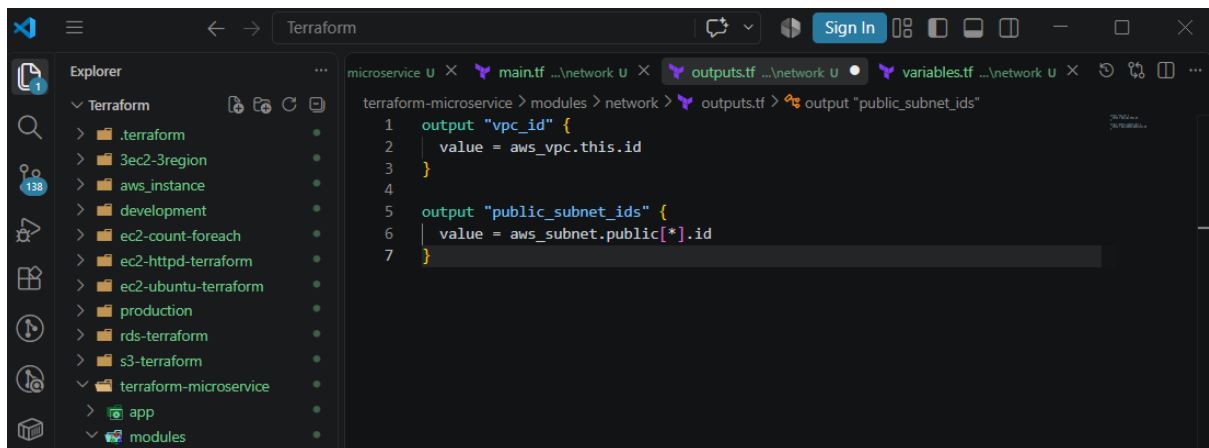
Create:

modules/network/variables.tf



Create:

modules/network/outputs.tf

A screenshot of a code editor window titled 'Terraform'. The left sidebar shows a file explorer with a tree structure: 'Terraform' (expanded) contains '.terraform', '3ec2-3region', 'aws_instance', 'development', 'ec2-count-foreach', 'ec2-httpd-terraform', 'ec2-ubuntu-terraform', 'production', 'rds-terraform', 's3-terraform', 'terraform-microservice', 'app', and 'modules'. The main editor area shows the file 'outputs.tf' with the following content:

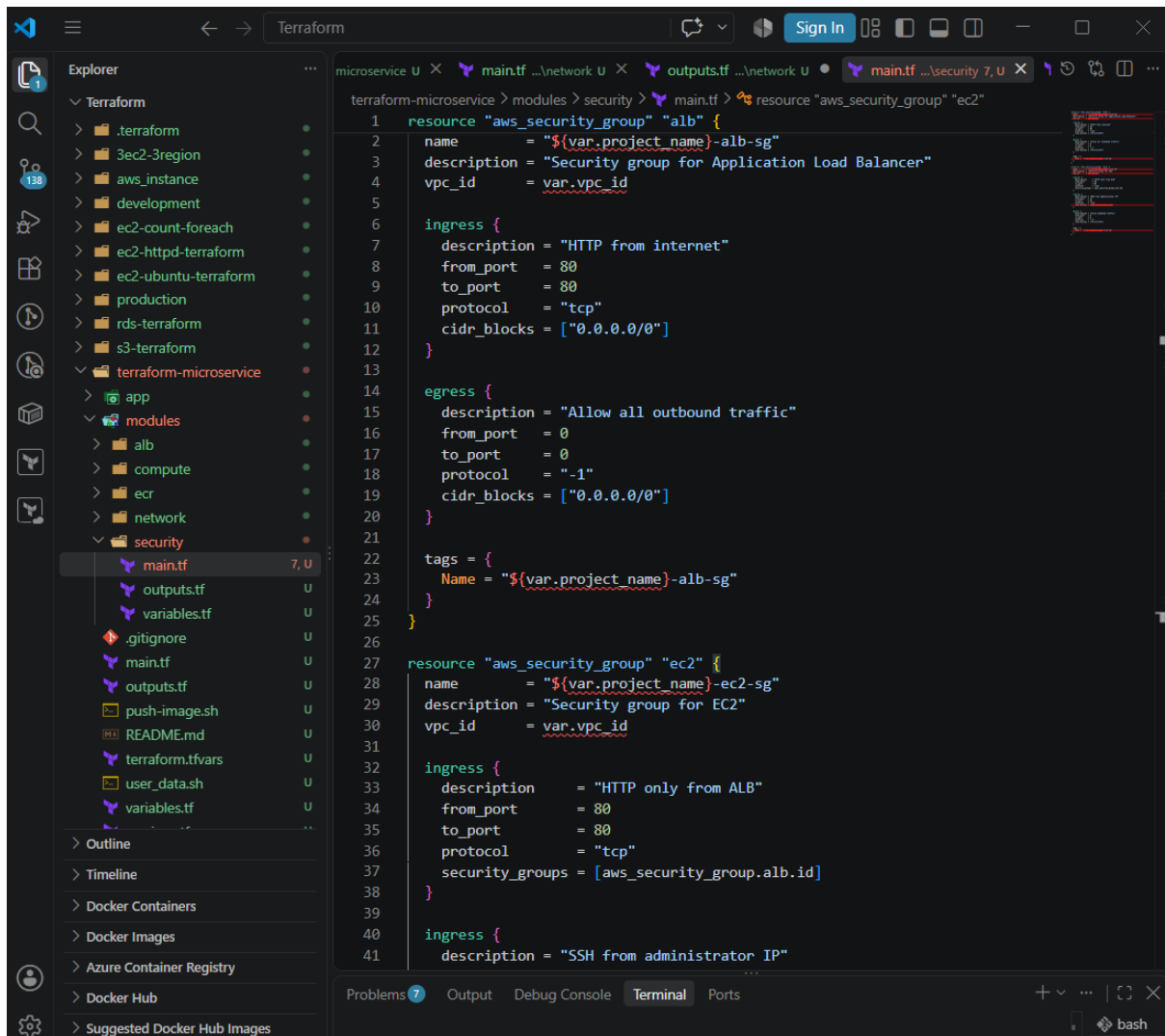
```
1 output "vpc_id" {  
2   value = aws_vpc.this.id  
3 }  
4  
5 output "public_subnet_ids" {  
6   value = aws_subnet.public[*].id  
7 }
```

12. Step 10 — Security module

This is one of the most important parts of the challenge.

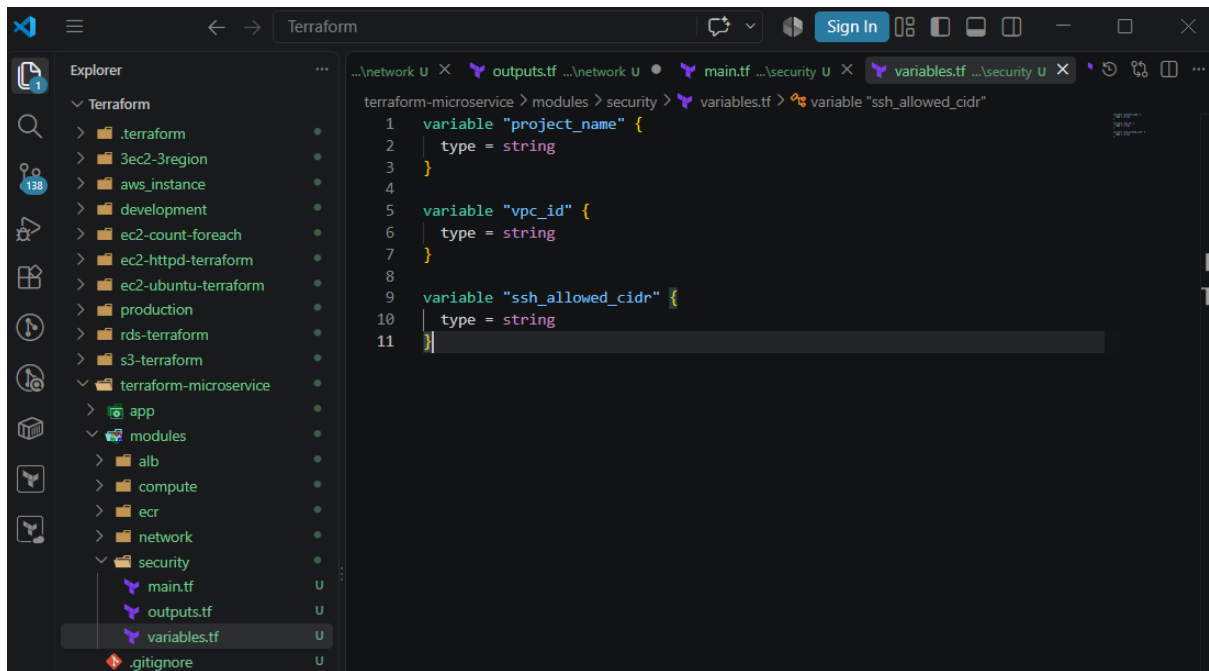
Create:

modules/security/main.tf



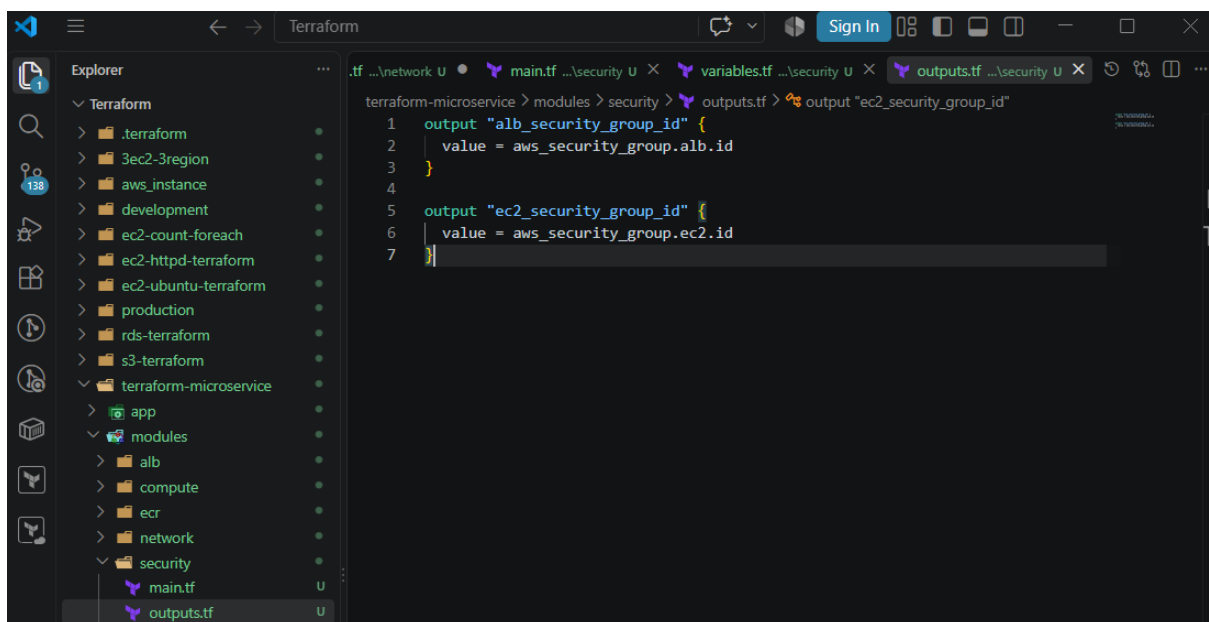
Create:

modules/security/variables.tf



Create:

modules/security/outputs.tf



Why this is secure

Notice what we **didn't** write:

```
cidr_blocks = ["0.0.0.0/0"]
```

for EC2 port 80.

Instead:

```
security_groups = [aws_security_group.alb.id]
```

Therefore:

Internet

|

X

|

EC2:80

is blocked.

But:

Internet

|

v

ALB:80

|

v

EC2:80

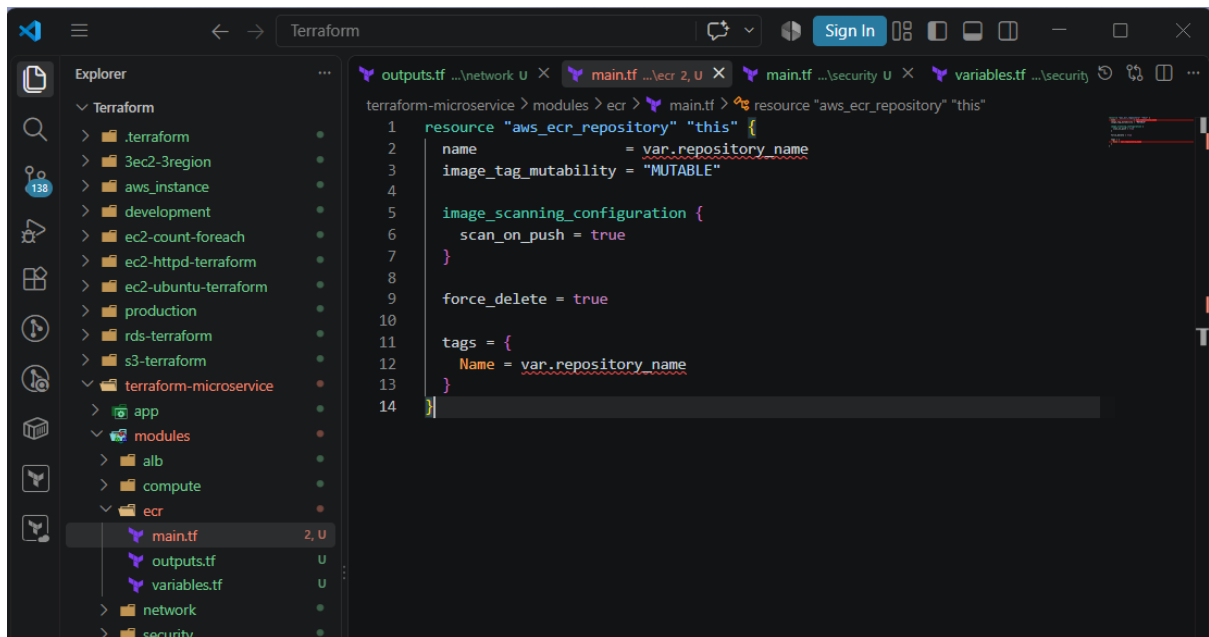
is allowed.

13. Step 11 — ECR module

AWS ECR is the Docker registry for this exercise. Terraform's AWS provider provides `aws_ecr_repository`, including its repository URI.

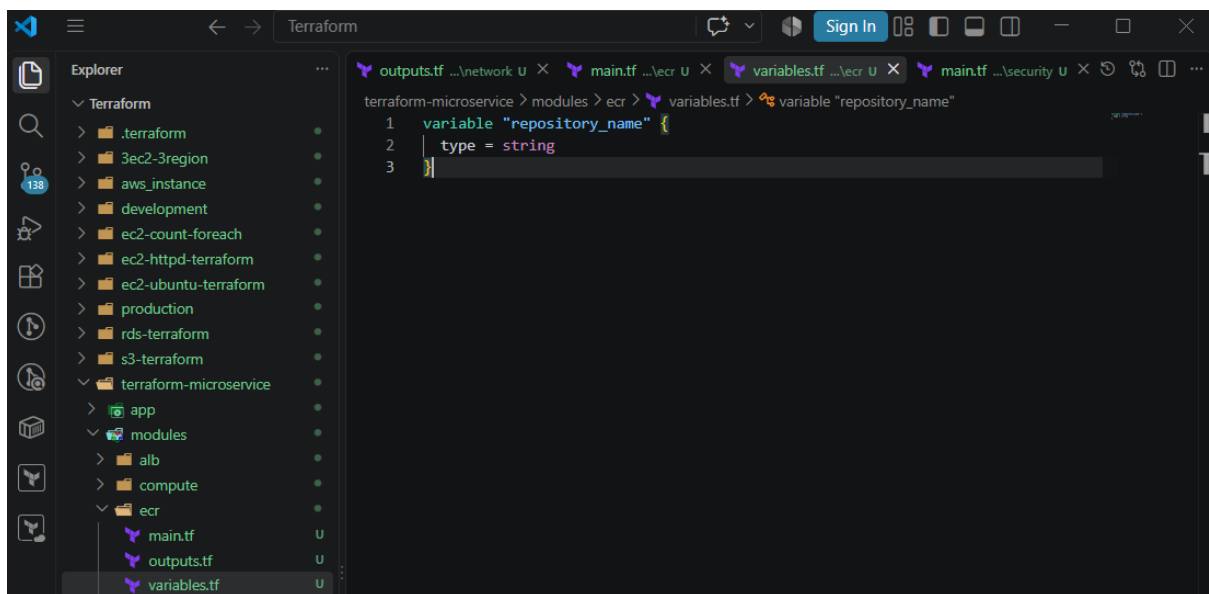
Create:

`modules/ecr/main.tf`



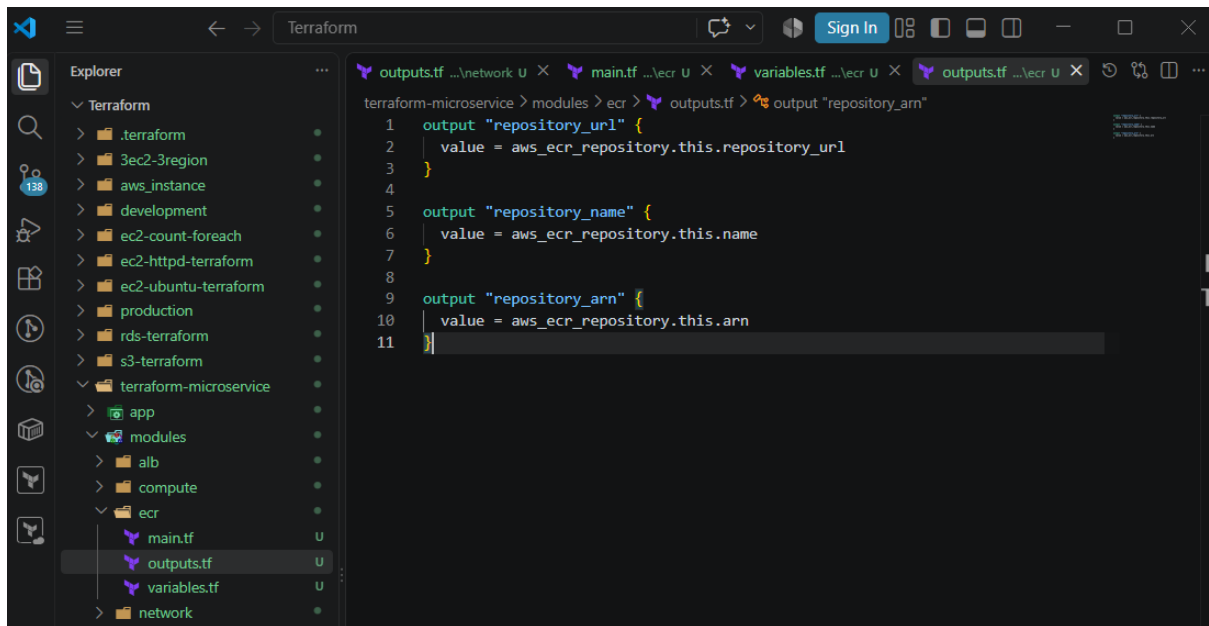
Create:

modules/ecr/variables.tf



Create:

modules/ecr/outputs.tf



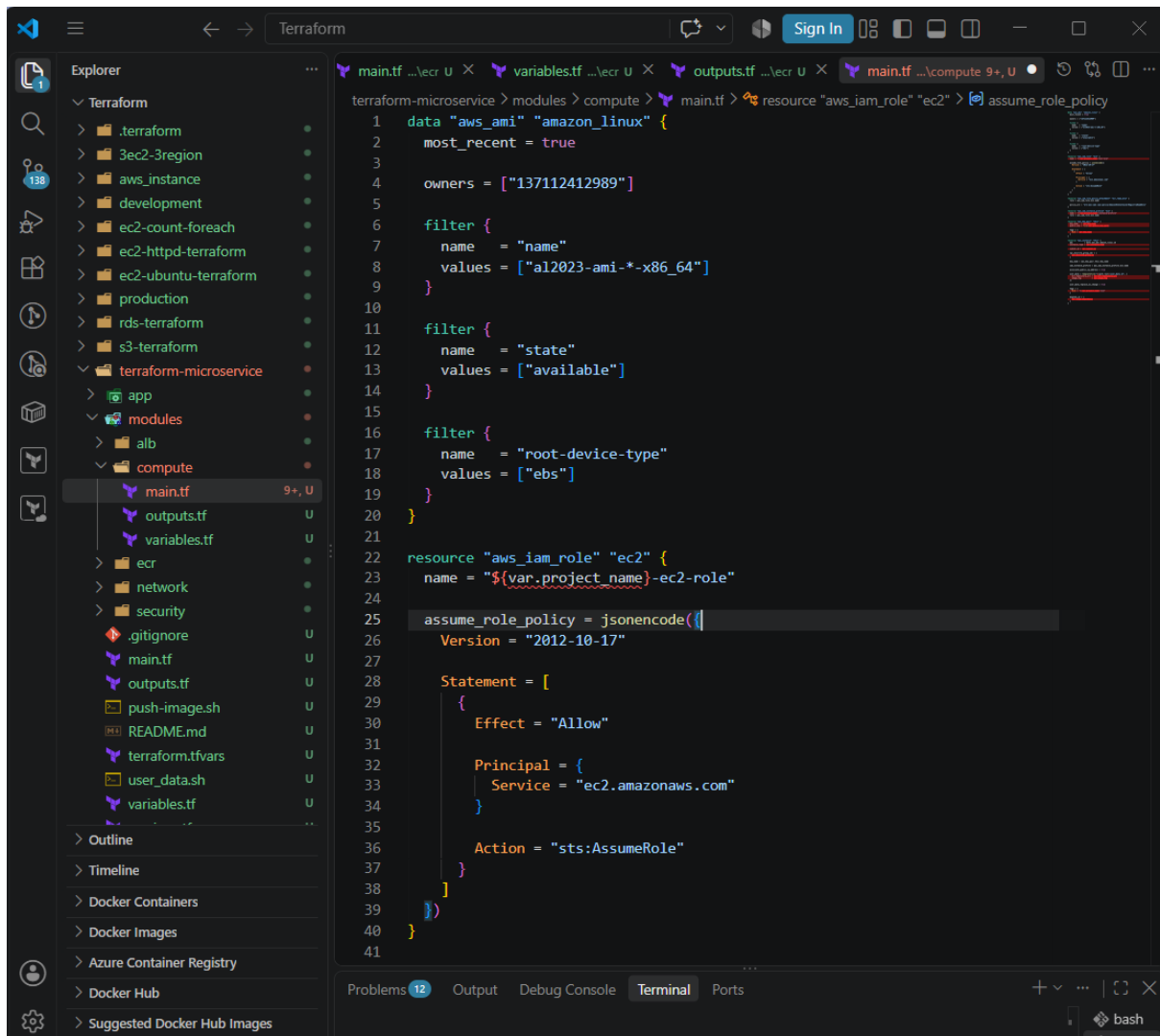
14. Step 12 — Compute module

Now we create EC2.

We will give the EC2 instance an IAM role allowing ECR image pulls. AWS provides managed ECR read-only permissions for this purpose.

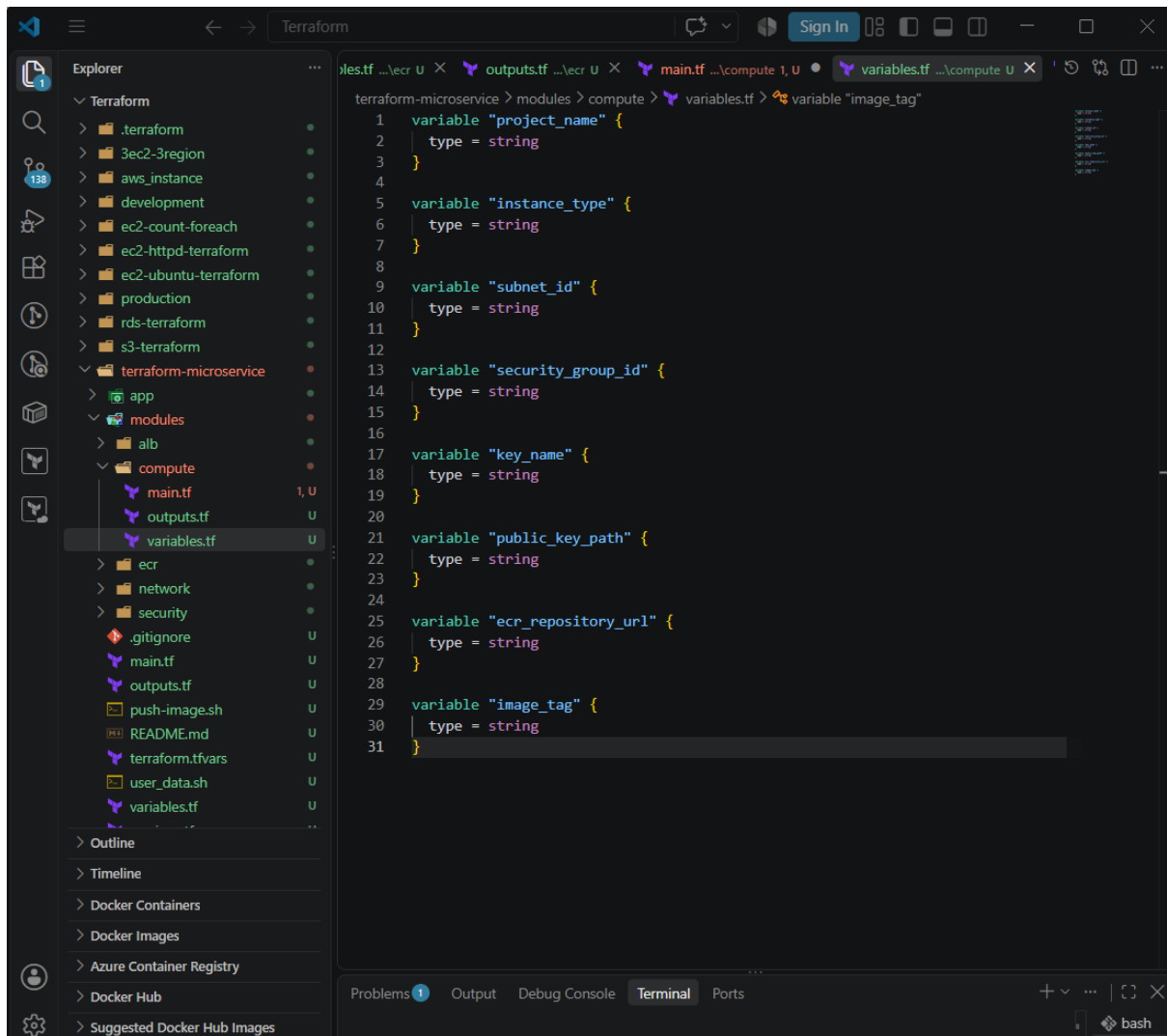
Create:

modules/compute/main.tf



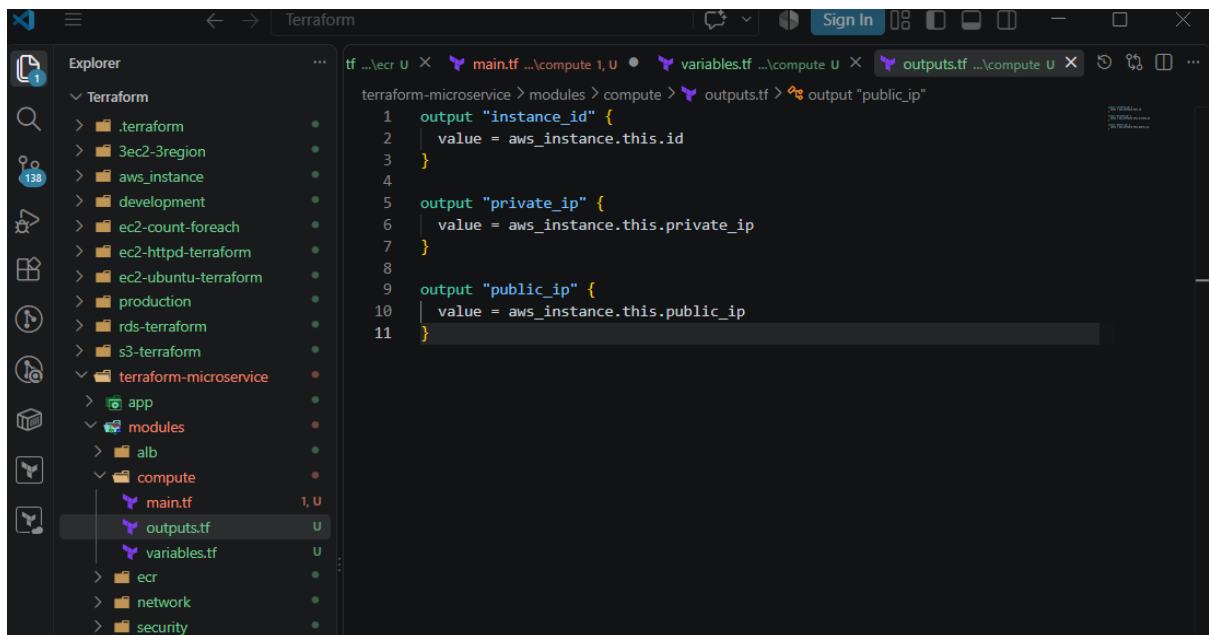
Create:

modules/compute/variables.tf



Create:

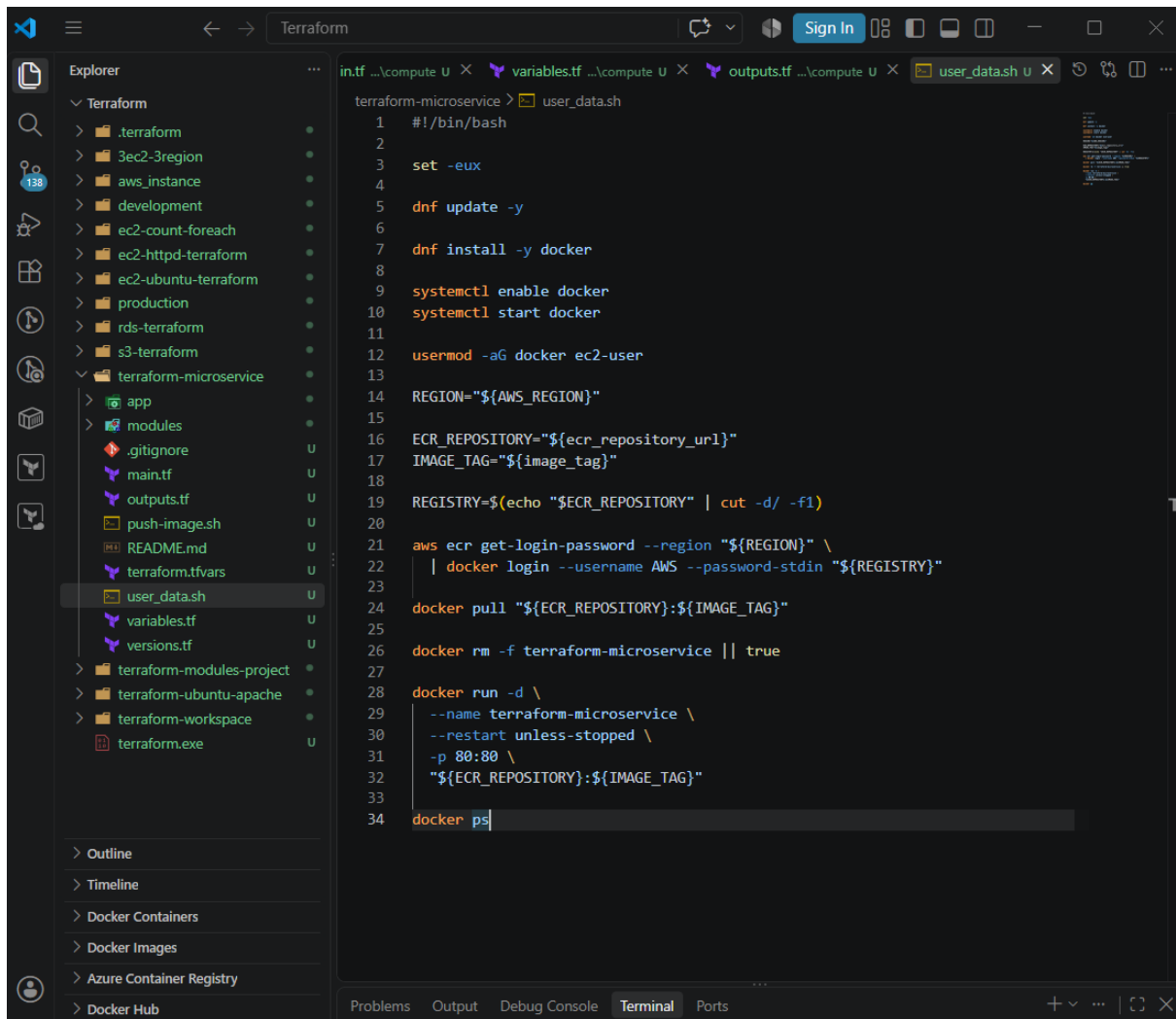
modules/compute/outputs.tf



15. Step 13 — EC2 user-data

Create:

user_data.sh



```
1  #!/bin/bash
2
3  set -eux
4
5  dnf update -y
6
7  dnf install -y docker
8
9  systemctl enable docker
10 systemctl start docker
11
12 usermod -aG docker ec2-user
13
14 REGION="${AWS_REGION}"
15
16 ECR_REPOSITORY="${ecr_repository_url}"
17 IMAGE_TAG="${image_tag}"
18
19 REGISTRY=$(echo "$ECR_REPOSITORY" | cut -d/ -f1)
20
21 aws ecr get-login-password --region "$REGION" \
22 | docker login --username AWS --password-stdin "$REGISTRY"
23
24 docker pull "${ECR_REPOSITORY}:${IMAGE_TAG}"
25
26 docker rm -f terraform-microservice || true
27
28 docker run -d \
29 --name terraform-microservice \
30 --restart unless-stopped \
31 -p 80:80 \
32 "${ECR_REPOSITORY}:${IMAGE_TAG}"
33
34 docker ps
```

There is one important correction here.

Because the EC2 AMI's region is not automatically available as an environment variable, we should pass the region from Terraform.

Change:

```
user_data = templatefile("${path.root}/user_data.sh", {
    ecr_repository_url = var.ecr_repository_url
    image_tag          = var.image_tag
})
```

to:

```
user_data = templatefile("${path.root}/user_data.sh", {
    aws_region      = var.aws_region
```

```
    ecr_repository_url = var.ecr_repository_url  
    image_tag          = var.image_tag  
  })
```

Add this variable:

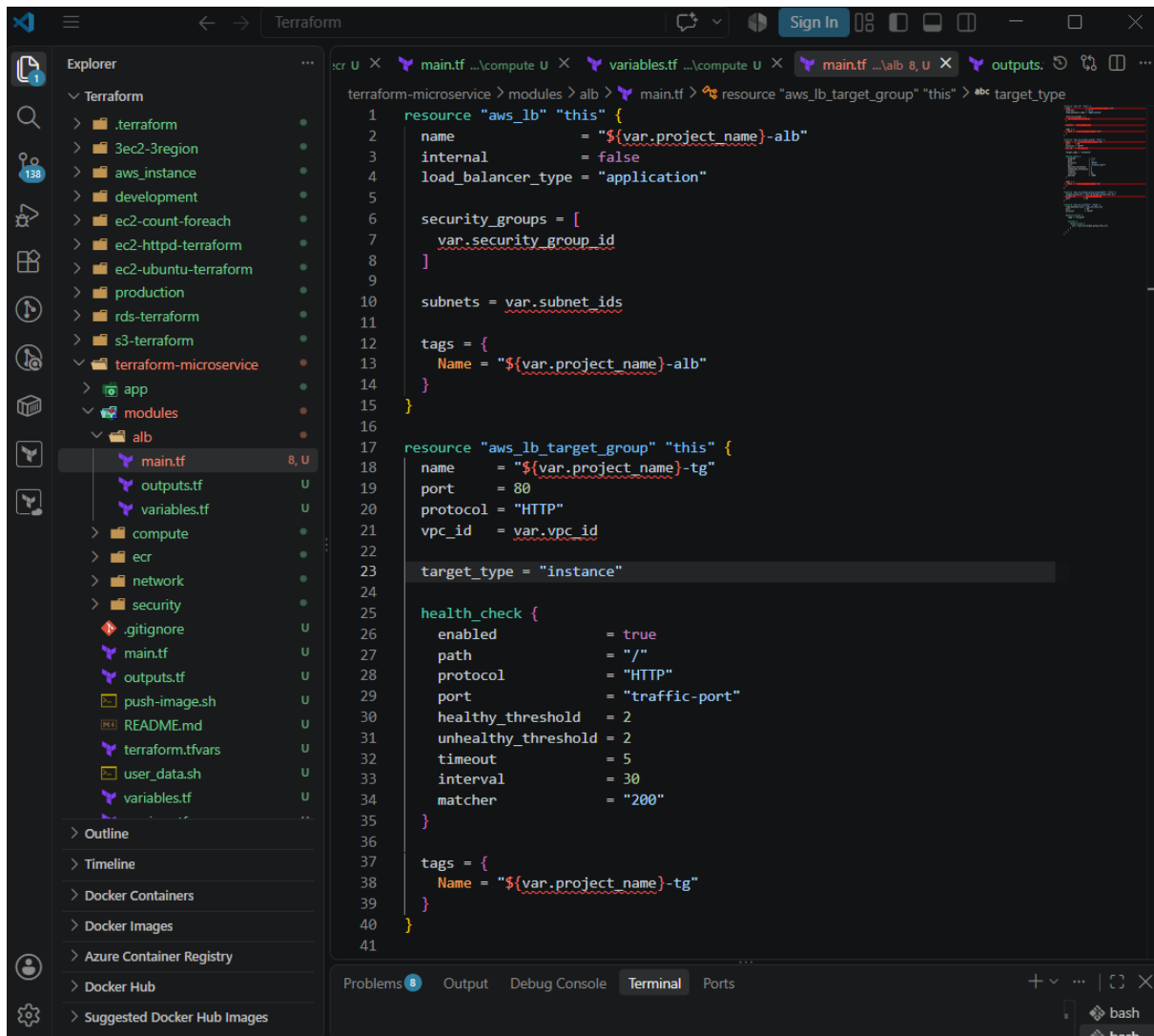
```
variable "aws_region" {  
  type = string  
}
```

And change:

```
REGION="${AWS_REGION}"
```

to:

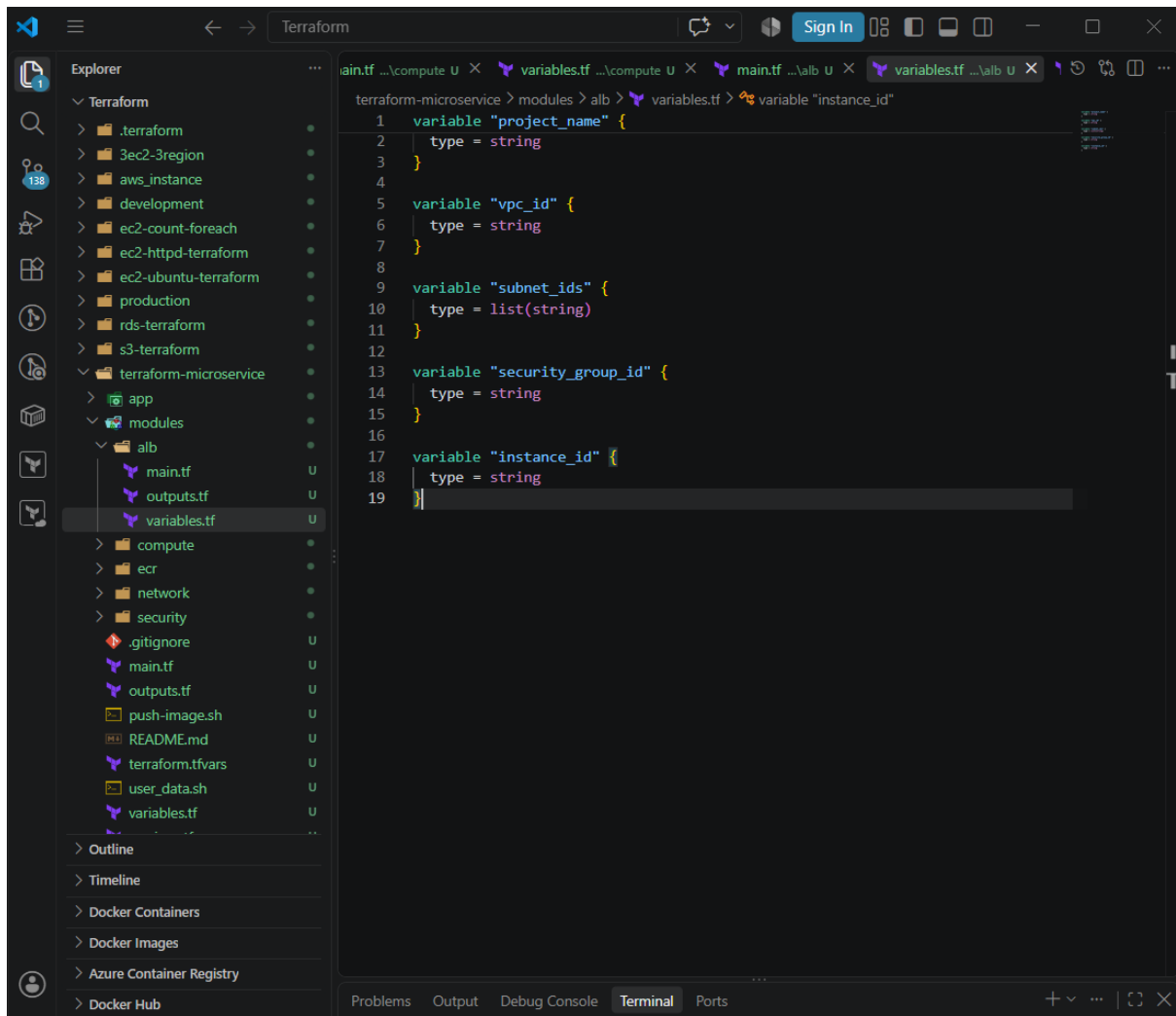
```
REGION="${aws_region}"
```



16. Step 14 — ALB module

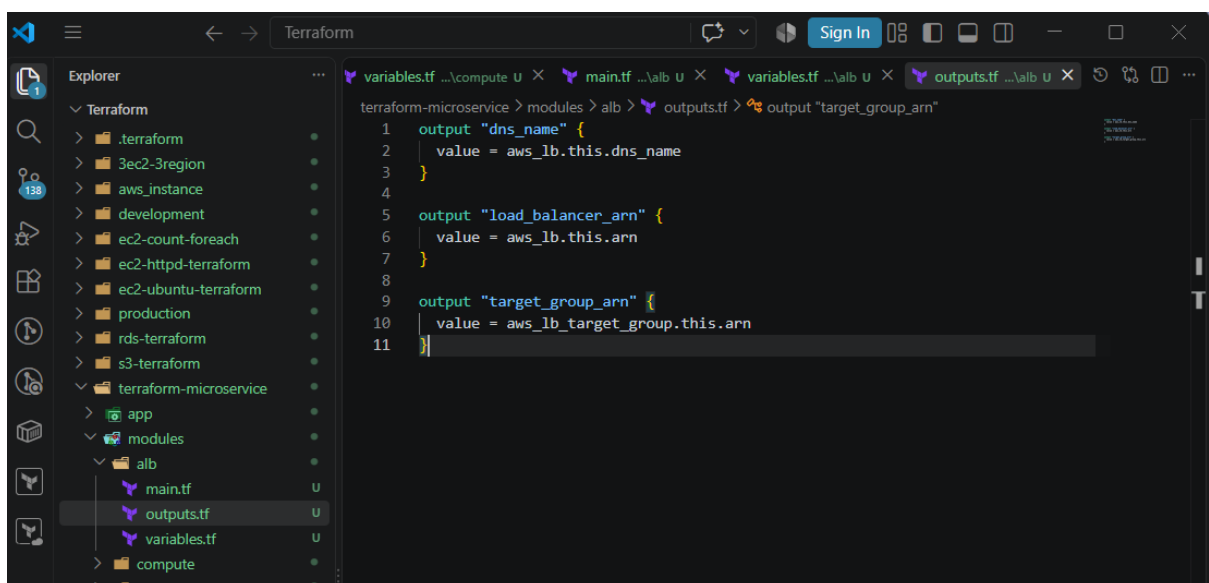
Create:

modules/alb/main.tf



Create:

modules/alb/outputs.tf

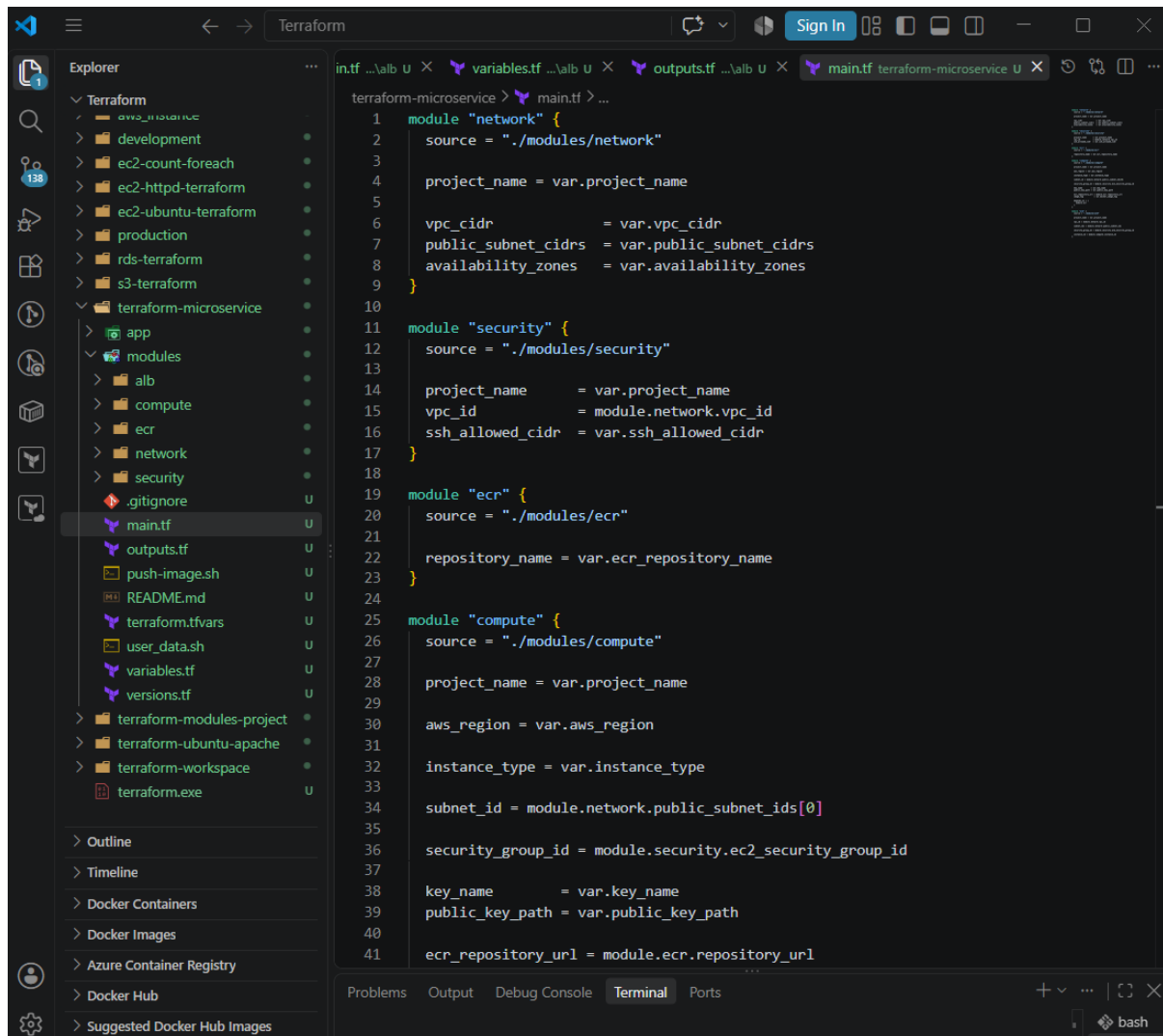


17. Step 15 — Root main.tf

Now connect all modules.

Create:

main.tf



18. Step 16 — Push the locally built image

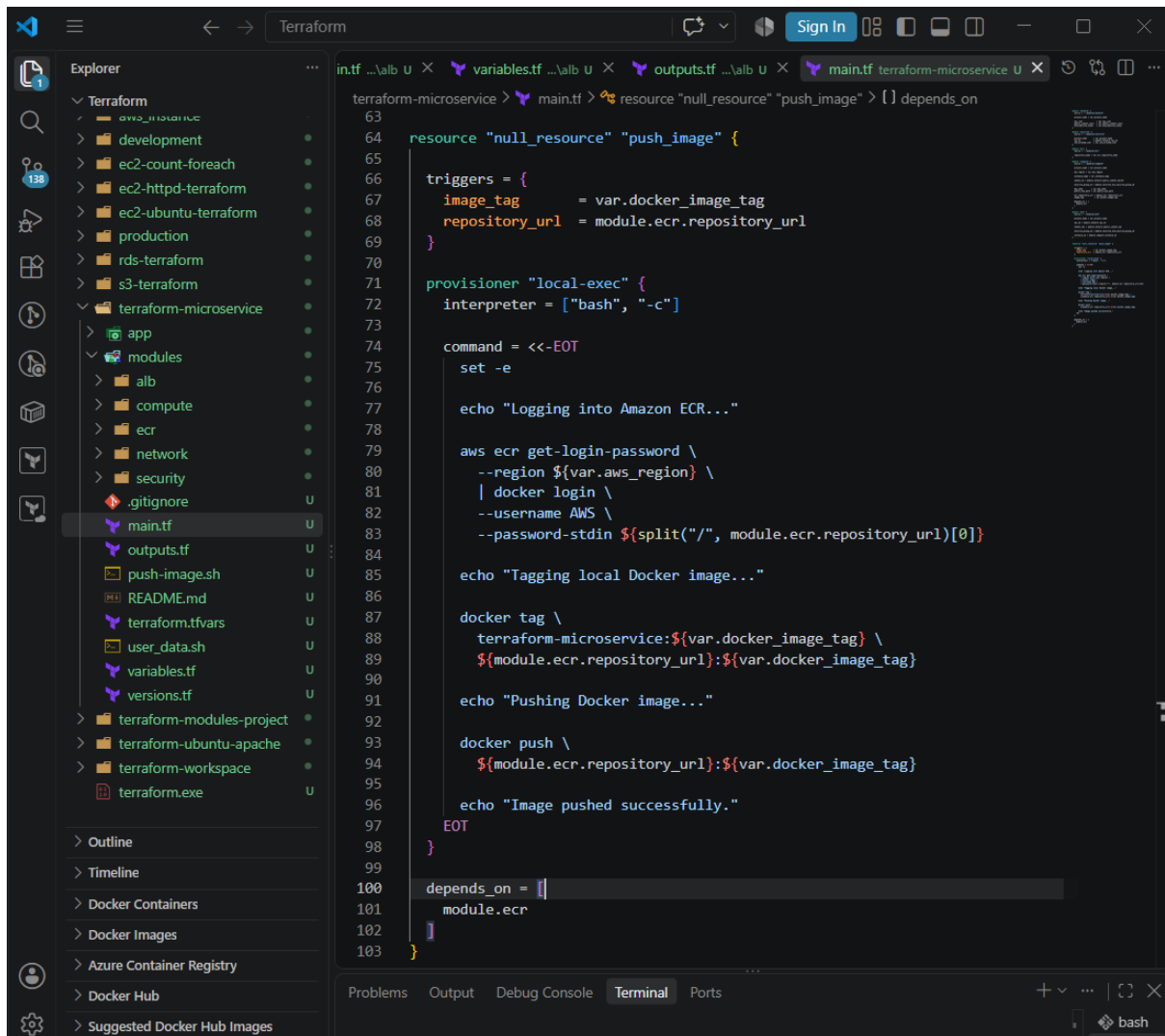
Now we need to satisfy this requirement:

Once the image registry is created, use Terraform to push the local image to the registry.

We'll use a Terraform `null_resource` and local Docker CLI.

This is actually a good solution for this assignment because the image was explicitly required to be built locally first.

Add the following to your **root main.tf** after the ECR module:



19. Important dependency

We want this order:

ECR

|

v

Push Docker image

|

v

EC2

|

v

Pull Docker image

|

v

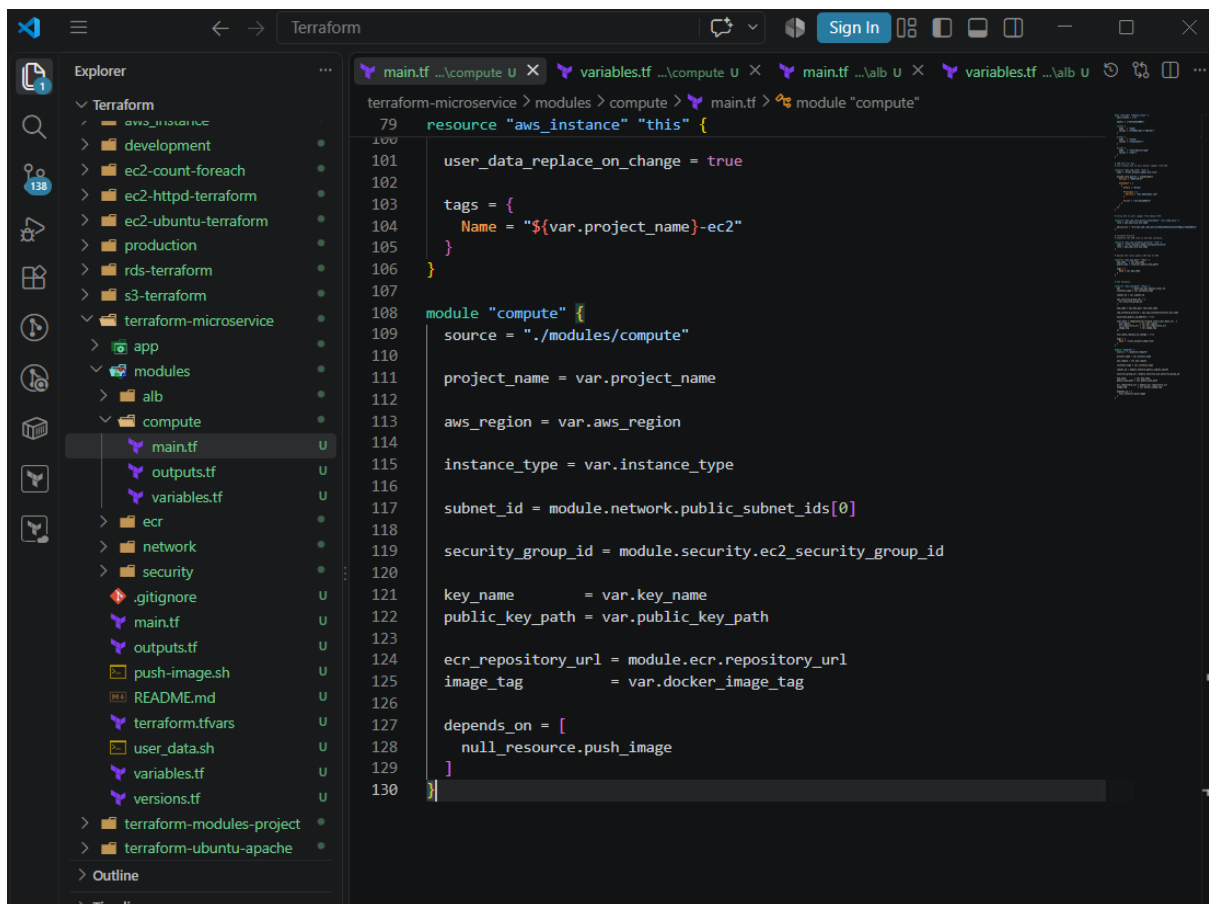
Container running

|

v

ALB

So modify the compute module block in main.tf:



```
79 resource "aws_instance" "this" {
100
101   user_data_replace_on_change = true
102
103   tags = {
104     Name = "${var.project_name}-ec2"
105   }
106 }
107
108 module "compute" {
109   source = "../modules/compute"
110
111   project_name = var.project_name
112
113   aws_region = var.aws_region
114
115   instance_type = var.instance_type
116
117   subnet_id = module.network.public_subnet_ids[0]
118
119   security_group_id = module.security.ec2_security_group_id
120
121   key_name      = var.key_name
122   public_key_path = var.public_key_path
123
124   ecr_repository_url = module.ecr.repository_url
125   image_tag          = var.docker_image_tag
126
127   depends_on = [
128     null_resource.push_image
129   ]
130 }
```

This is important.

Terraform now knows:

ECR must exist

↓

Docker image must be pushed



EC2 can be created

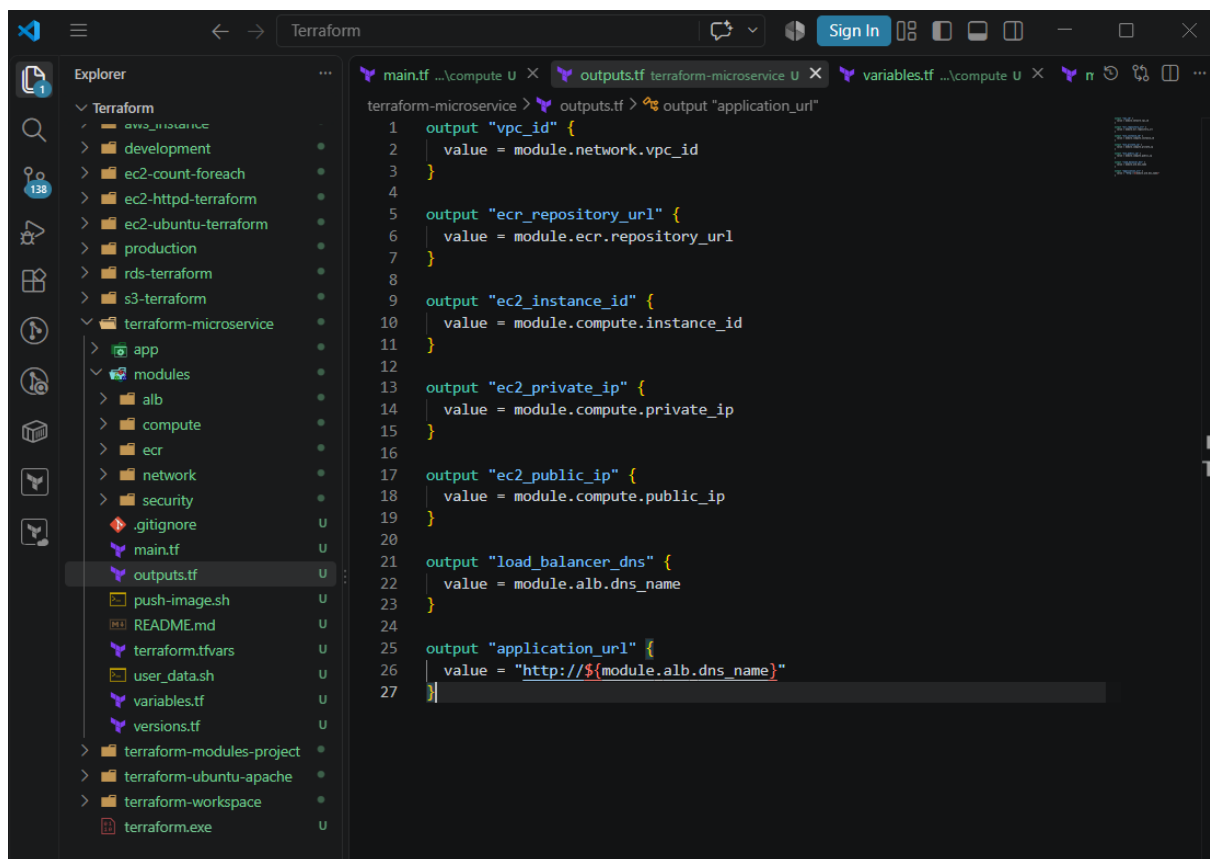


EC2 pulls image

20. Step 17 — Outputs

Create:

outputs.tf



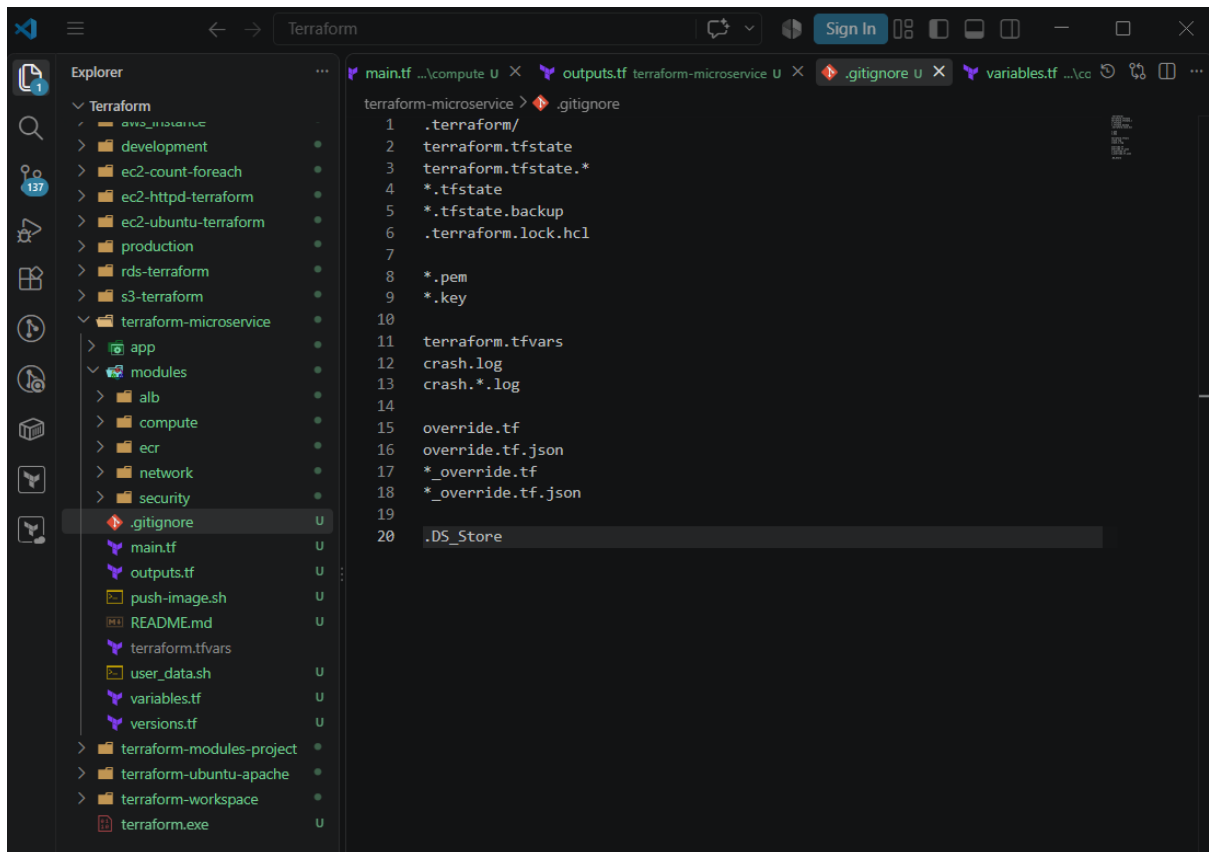
The screenshot shows the Visual Studio Code interface with the Terraform project. The Explorer on the left lists the project files, including modules for alb, compute, ecr, network, security, and others. The main editor displays the contents of the outputs.tf file, which defines several Terraform outputs for the terraform-microservice module.

```
1 output "vpc_id" {
2   | value = module.network.vpc_id
3 }
4
5 output "ecr_repository_url" {
6   | value = module.ecr.repository_url
7 }
8
9 output "ec2_instance_id" {
10  | value = module.compute.instance_id
11 }
12
13 output "ec2_private_ip" {
14   | value = module.compute.private_ip
15 }
16
17 output "ec2_public_ip" {
18   | value = module.compute.public_ip
19 }
20
21 output "load_balancer_dns" {
22   | value = module.alb.dns_name
23 }
24
25 output "application_url" {
26   | value = "http://${module.alb.dns_name}"
27 }
```

21. Step 18 — .gitignore

Create:

.gitignore



important

You normally **should commit** **.terraform.lock.hcl** to Git for reproducible provider versions. So if you're treating this as a portfolio/assignment repository, remove:

`.terraform.lock.hcl`

from `.gitignore`.

The better `.gitignore` is therefore:

22. Step 19 — Verify AWS CLI

Before running Terraform:

```
aws sts get-caller-identity
```

You should get something similar to:

```
MINGW64:/c/Terraform/terraform-microservice
brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ aws sts get-caller-identity
{
  "UserId": "820982384063",
  "Account": "820982384063",
  "Arn": "arn:aws:iam::820982384063:root"
}

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ aws configure
AWS Access Key ID [*****35G0]: AKIA36JS5AW77HBT35G0
AWS Secret Access Key [*****shtc]: vQ05ocVcAoivAE7hoqqiw4nArsqLNKQKlT
swhstc
Default region name [us-east-1]:
Default output format [json]:

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ aws sts get-caller-identity
{
  "UserId": "820982384063",
  "Account": "820982384063",
  "Arn": "arn:aws:iam::820982384063:root"
}

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ |
```

23. Step 20 — Verify Docker

Terraform will use the Docker daemon on your local machine.

Run:

```
docker version
```

Then:

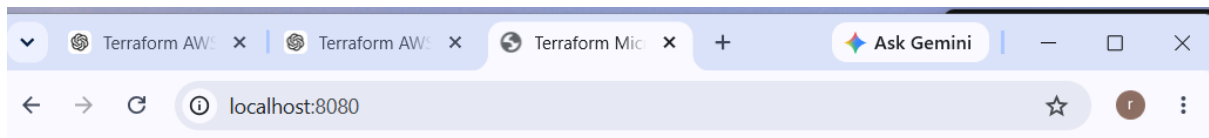
```
docker images
```

You must have:

```
terraform-microservice:latest
```

If you don't:

```
docker build -t terraform-microservice:latest ./app
```



Hello Rajkumari...from Terraform!

Docker + Nginx + AWS ECR + EC2 + ALB

This application was deployed using Terraform.

24. Step 21 — Terraform initialization

From:

```
terraform-microservice/
```

run:

```
terraform init
```

You should see providers being downloaded.

Then:

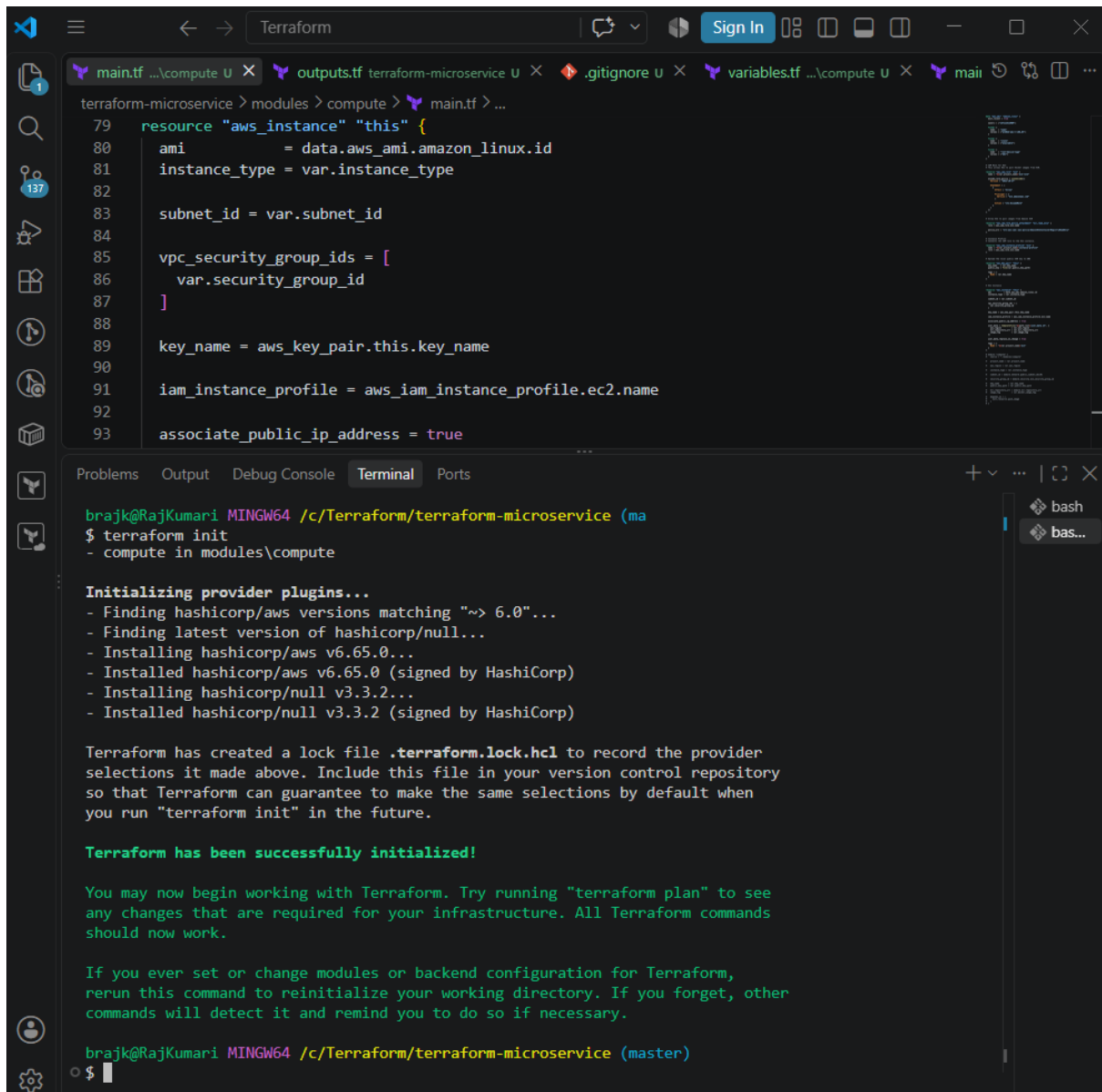
```
terraform fmt -recursive
```

Then:

```
terraform validate
```

Expected:

Success! The configuration is valid.



The screenshot shows a Visual Studio Code editor window with a Terraform project. The top pane displays the `main.tf` file within the `modules > compute > main.tf` path. The code defines an `aws_instance` resource named `this` with the following configuration:

```
resource "aws_instance" "this" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  subnet_id = var.subnet_id

  vpc_security_group_ids = [
    var.security_group_id
  ]

  key_name = aws_key_pair.this.key_name

  iam_instance_profile = aws_iam_instance_profile.ec2.name

  associate_public_ip_address = true
}
```

The bottom pane shows the `Terminal` output, which displays the successful execution of `terraform init` in the `modules/compute` directory. The output includes the initialization of provider plugins for HashiCorp's AWS and Null providers, the creation of a `.terraform.lock.hcl` file, and a confirmation message: **Terraform has been successfully initialized!**

25. Step 22 — Terraform plan

Run:

`terraform plan`

You should see resources similar to:

`aws_vpc`

`aws_internet_gateway`

`aws_subnet`

`aws_route_table`

`aws_route_table_association`

aws_security_group
aws_ecr_repository
aws_iam_role
aws_iam_role_policy_attachment
aws_iam_instance_profile
aws_key_pair
aws_instance
aws_lb
aws_lb_target_group
aws_lb_target_group_attachment
aws_lb_listener
null_resource.push_image

Review the plan carefully.

26. Step 23 — Terraform apply

Run:

```
terraform apply
```

Terraform will ask:

Do you want to perform these actions?

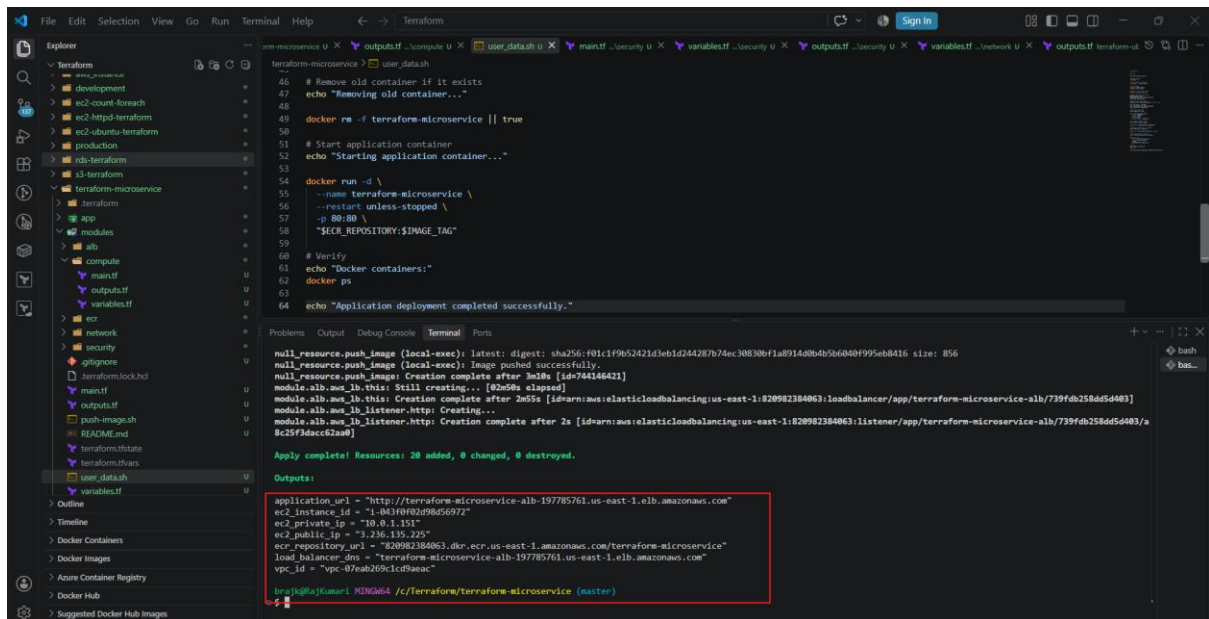
Enter:

Yes

27. Step 24 — Check Terraform outputs

Run:

```
terraform output
```



```
46 # Remove old container if it exists
47 echo "Removing old container..."
48
49 docker rm -f terraform-microservice || true
50
51 # Start application container
52 echo "Starting application container..."
53
54 docker run -d \
55   --name terraform-microservice \
56   --restart unless-stopped \
57   -p 80:80 \
58   "ECR_REPOSITORY:$IMAGE_TAG"
59
60 # Verify
61 echo "Docker containers:"
62 docker ps
63
64 echo "Application deployment completed successfully."
```

Problems Output Debug Console Terminal Ports

null_resource.push_image (local-exec): latest: digest: sha256:f01c1f9b52421d3eb1d244287b74ac30830bf1a8914d0b4b5b6040f995eb8416 size: 856
null_resource.push_image (local-exec): Image pushed successfully.
null_resource.push_image: Creation complete after 3m0s [id=744146421]
module.alb.aws_lb.this: Still creating... [02m0s elapsed]
module.alb.aws_lb.this: Creation complete after 2m55s [id=arn:aws:elasticloadbalancing:us-east-1:820982384063:loadbalancer/app/terraform-microservice-alb/739fdb258dd5d4083]
module.alb.aws_lb_listener.http: Creating...
module.alb.aws_lb_listener.http: Creation complete after 2s [id=arn:aws:elasticloadbalancing:us-east-1:820982384063:listener/app/terraform-microservice-alb/739fdb258dd5d4083/a8c25f3dacc62aa8]

Apply complete! Resources: 20 added, 0 changed, 0 destroyed.

Outputs:

application_url = "http://terraform-microservice-alb-197785761.us-east-1.elb.amazonaws.com"
ec2_instance_id = "i-043f0f02d98d56972"
ec2_private_ip = "10.0.1.151"
ec2_public_ip = "3.236.135.225"
ecr_repository_url = "820982384063.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice"
load_balancer_dns = "terraform-microservice-alb-197785761.us-east-1.elb.amazonaws.com"
vpc_id = "vpc-07eab209c1cd9a5ac"

braj@ajkumar1 MINGW64 /c/Terraform/terraform-microservice (master)

28. Step 25 — Test the application

Get the ALB URL:

terraform output -raw application_url

terraform.tfvars terraform-microservice X main.tf ...network U X outputs.tf ...network U X main.tf ...ecr U X vari: 137

terraform-microservice > terraform.tfvars > [] availability_zones > abc 1

11
12 availability_zones = [
13 "us-east-1a",
14 "us-east-1b"
15]
16
17 instance_type = "t3.micro"
18
19 ssh_allowed_cidr = "117.192.67.251/32"
20
21 key_name = "terraform-microservice-key"
22 public_key_path = "C:/Users/braj/Downloads/terraform-microservice-key.pub"
23
24 ecr_repository_name = "terraform-microservice"
25
26 docker_image_tag = "latest"

Problems Output Debug Console Terminal Ports

bash
bash

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
\$ aws ec2 describe-instances \
> --instance-ids i-0788eb94176dacf2d \
> --query 'Reservations[0].Instances[0].SecurityGroups[*].GroupId' \
● > --output text
sg-01e9c0add3d2badd2

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
\$ aws ec2 describe-instances \
> --instance-ids i-0788eb94176dacf2d \
> --query 'Reservations[0].Instances[0].[PublicIpAddress,PrivateIpAddress,State.Name]' \
● > --output table

|DescribeInstances|
+-----+
| 44.205.18.237 |
| 10.0.1.219 |
| running |
+-----+

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
\$

```

ec2-user@ip-10-0-1-144:~
[ec2-user@ip-10-0-1-144 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-10-0-1-144 ~]$ docker ps
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS        PORTS
c955225bbdd3   820982384063.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:latest "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  0.0.0.0:80->80/tcp, :::80->80/tcp
[ec2-user@ip-10-0-1-144 ~]$ docker images
REPOSITORY      TAG
IMAGE ID        CREATED        SIZE
820982384063.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice   latest
5bcb9af4a9bc    44 hours ago  69MB
[ec2-user@ip-10-0-1-144 ~]$ aws ecr describe-images \
--repository-name terraform-microservice \
--region us-east-1 \
--query 'imageDetails[*].[imageTags,imagePushedAt]' \
--output table
Row should have 2 elements, instead it has 1
[ec2-user@ip-10-0-1-144 ~]$ curl http://terraform-microservice-alb-197785761.us-east-1.elb.amazonaws.com
<!DOCTYPE html>
<html>
<head>
  <title>Terraform Microservice</title>
</head>
<body>
  <h1>Hello Rajkumari....from Terraform!</h1>
  <h2>Docker + Nginx + AWS ECR + EC2 + ALB</h2>
  <p>This application was deployed using Terraform.</p>
</body>
[ec2-user@ip-10-0-1-144 ~]$ |

```

```

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ aws elbv2 describe-target-health \
> --target-group-arn "arn:aws:elasticloadbalancing:us-east-1:820982384063:targetgroup/terraform-microservice-tg/21eea8e4cfe316ed" \
> --query 'TargetHealthDescriptions[*].[Target.Id,Target.Port,TargetHealth.State,TargetHealth.Reason]' \
> --output table
-----
| DescribeTargetHealth |
+-----+-----+-----+
| i-031f7957ef5ad4fa9 | 80 | healthy | None |
+-----+-----+-----+

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ |

```

```

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ aws elbv2 describe-target-health \
> --target-group-arn "arn:aws:elasticloadbalancing:us-east-1:820982384063:targetgroup/terraform-microservice-tg/21eea8e4cfe316ed" \
> --query 'TargetHealthDescriptions[*].[Target.Id,Target.Port,TargetHealth.State,TargetHealth.Reason]' \
> --output table
-----
| DescribeTargetHealth |
+-----+-----+-----+
| i-031f7957ef5ad4fa9 | 80 | healthy | None |
+-----+-----+-----+

brajk@RajKumari MINGW64 /c/Terraform/terraform-microservice (master)
$ |

```

Hello Rajkumari....from Terraform!

Docker + Nginx + AWS ECR + EC2 + ALB

This application was deployed using Terraform.